

User Manual

Overview

What Is Tier0?

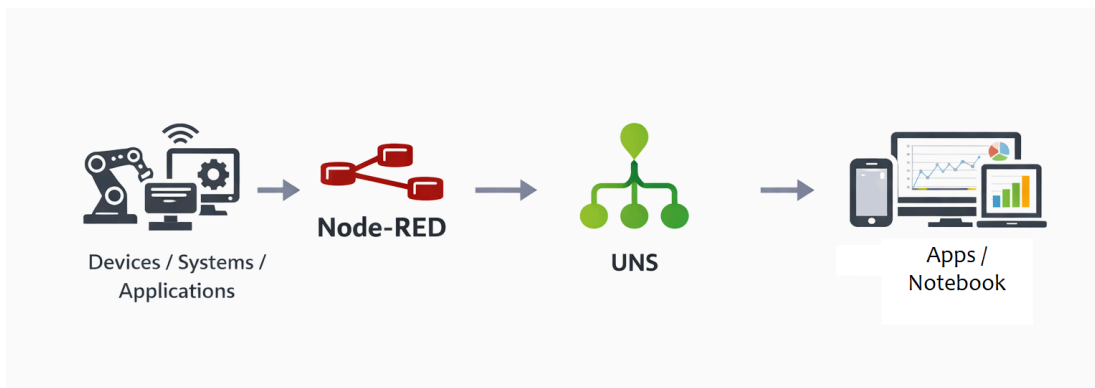
Tier0 is a Unified Namespace (UNS)-based industrial data platform.

It connects data through Node-RED, organizes it into a unified structure, and makes it available for applications and analytics.

How It Works?

Tier0 moves and structures data in a single workflow:

- Node-RED connects and processes data from devices and systems
- Data is published into the Unified Namespace (UNS), structured and stored in Tier0 database
- Data is consumed by dashboards, applications and analytics



Key Components

Unified Namespace (UNS)

Defines a hierarchical structure for all industrial data.

```
factory/workshopA/line1/cnc01/telemetry/spindleSpeed
```

Namespace

factory / workshopA / line1 / cnc01 / telemetry / Metric / spindleSpeed

ImportExport

TopicTemplateLabel

Please enter name/alias/path

+

factory(1)

workshopA(1)

line1(1)

cnc01(1)

telemetry(1)

Metric(1)

spindleSpeed

spindleSpeed

Details

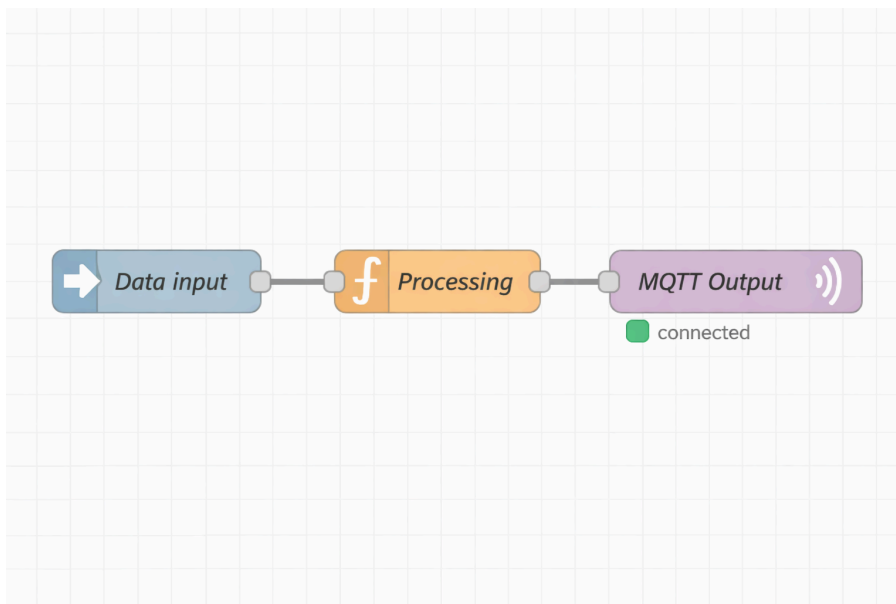
Definition

Attribute	Type	Length	Display Name	Remark
timeStamp	DATETIME			
speed	INTEGER			
quality	LONG			

Node-RED Integration

The core engine for data connection and processing.

- Connect devices, applications, and systems
- Format and publish data into the namespace



Python Notebook

Analyze data with full flexibility using Python with an interactive environment.



Agentic App Builder

Create industrial applications using natural language without consuming any of your local resources. The whole process is done on cloud, and provide a package for local installation.



Homepage Introduction

TIER  Enterprise

UNSDev ToolsSystemNamespace

1

2

3

ImportExport

Namespace

TopicTemplateLabel4

v1(6)

Plant_Name(6)5

SMT-Area-01(6)

SMT-Line-01(6)

Printer-Cell(6)

Printer01(6)

Action(2)

start_job

stop_job

Metric(2)

board_cycle...

boards_count

State(2)

alarm_status

current_job

Overview6

Message In Rate

0 msg/s

Message Out Rate

0 msg/s

All MQTT Connections

6

Live MQTT Connections

6

Functions7

Source Flow

Connect multi-source data.

Event Flow

Process data and create data-based event s.

Dashboards

Quick view of key metrics and insights.

MQTT Access ⓘ8

MQTT URL

mqtt://192.168.31.167:1...

MQTT Port

1883

Topic

Please select

Payload

Please select a topic.

Homepage

No.	Name	Description
1	Menu Bar	Lists the function menus of the platform, enabling quick access to various function modules.
2	Tool Bar	Common operations on the platform, including search and user information.
3	Import/Export	Integrate the data import/export function from Namespace/Source Flow/Event Flow/Dashboard together.
		Go back to the homepage.
4	Topic	Tree view of data models already built in UNS
	Template	Tree view of data model templates already built in UNS
	Label	Displays labels, and models under each label
5		Displays data models by category: Text/Template/Label .
	 / 	Search data models / Refresh data model list.
		Create folder-file structured data models based on data hierarchy.
6	Overview	Number of connections between data sources and the internal MQTT broker and messages in/out.
7	Functions	Lists all function modules of the platform.
8	MQTT Access	Detailed information of the internal MQTT broker. Select existing topics to see detailed payload.

Function Module Description

Module	Name	Description
UNS	Namespace (homepage)	Models industrial data into a hierarchical structure (folder-file) for clarity.
	Source Flow	Uses Node-RED to define the data source of the created model.
	Event Flow	Process Namespace data and configure data events via Node-RED.
	Dashboards	Displays data from Namespace on dashboards according to specific design.
Dev Tools	Containers	Integrates Portainer to manage docker containers.
	Notebook	Adopts Marimo for easy python script usage. For example, train models, run SQL queries and others.
	Advanced Use	Provides account and password of integrated application for deeper exploration.
System	Routing Management	Customizes Tier0 function module routing.
	User Management	Manage access control via users and roles.
	Open Data	Provide different types of data subscription, including API, MQTT and MCP.
	Menu Config	Supports menu changes on all existing function modules.

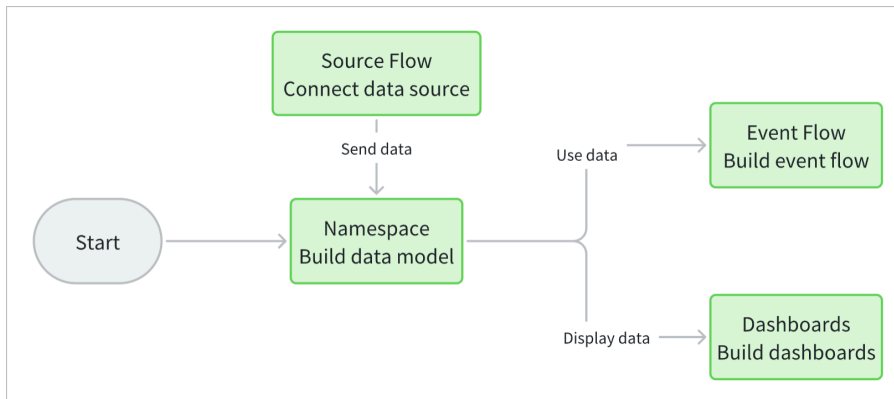
① info

- Private deployment. For detailed specifications and steps, see [Deploy Tier0](#).
- Applications built through natural language in **App Builder** is generated on cloud, and a package is available for local installation.

User Journey

User Journey

Follow the order displayed in the image, you will complete your first data flow in Tier0.



- Tier0 builds data models in a folder-file structure, making it easy to track and understand;
- It collects various types of data from diverse sources in **Source Flow**;
- Data collected to Tier0 can be processed in many ways in **Event Flow**;
- Processed data can be displayed on dashboards for better understanding through both **Event Flow** and **Dashboards**.
- Data collected to and processed in Tier0 can be obtained by third parties through [Open API](#) or MQTT subscription.

① **note**

- **Source Flow** and **Event Flow** are based on **Node-RED**. For detailed instructions, please see [NodeRed](#).
- **Dashboards** is based on **Grafana**. For detailed instructions, please see [Grafana](#).

Quick Start Guide

Deploy Tier0

System Requirements

Configuration Type	CPU	Memory	HDD/IOPS
Basic Configuration	4 cores	8 GB	100 GB, 2000 IOPS (30% write)
Recommended Configuration	8 cores	16 GB	1 TB, 2000 IOPS (30% write)

Installing Tier0

Linux

1. Make sure you have the following system and components ready.
 - Ubuntu Server 24.04
 - Docker

Component	Version
Docker Engine - Community	27.4.0
Docker Buildx	v0.19.2
Docker Compose	v2.31.0
Containerd	1.7.24

 note

Above system and components have been tested. You can try on other versions if you are interested.

2. Clone the project from Github.

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

3. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

4. Enter the **.env.default** file and press the **i** key to edit configuration items as needed.

```
vi .env.default
```

Item	Description
VOLUMES_PATH	The storage path for project data.
ENTRANCE_DOMAIN	Tier0 access address. Normally the IP or domain of the server where you install Tier0. ⚠ warning Do not use 127.0.0.1 or localhost, otherwise login and authentication functions will NOT work.
LANGUAGE	Tier0 display language. Chinese (zh-CN) and English (en-US) are supported.

```
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=linux

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=1
OS_NAME=Tier0
VOLUMES_PATH=/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work)
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192.168.1.100
ENTRANCE_PORT=8088
ENTRANCE_SSL_PORT=8443

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US
```

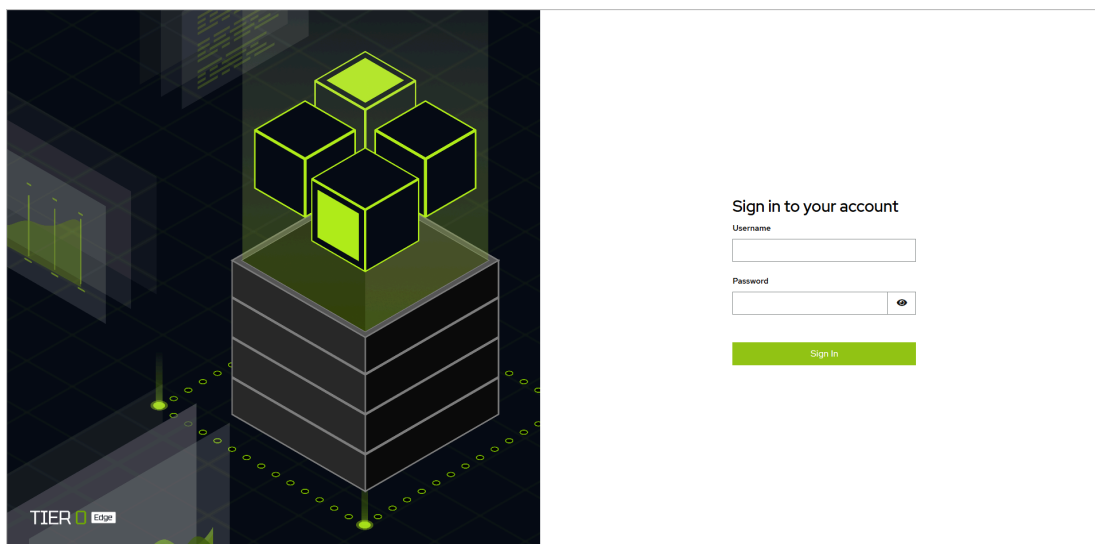
5. Press **ESC** to stop editing, save the file and install Tier0.

```
:wq!
bash bin/install.sh
```

6. Select whether to keep the entered entrance address and whether to use default setup, and then wait for the installation to complete.

```
Available options for ENTRANCE_DOMAIN:
0). Keep current: office.unibutton.com (default)
1). 192.168.31.167
2). 172.17.0.1
3). 172.18.0.1
Select option (0-3), or press Enter to keep current: 0
Keeping current ENTRANCE_DOMAIN: office.unibutton.com
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /root/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
```

7. Access Tier0 on a browser and log in.



Windows

info

- Make sure you have installed the latest **Docker Desktop** and **Git** on Windows 10 or 11.
- Docker must be started.

1. Clone the project from Github in **Git Bash**.

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

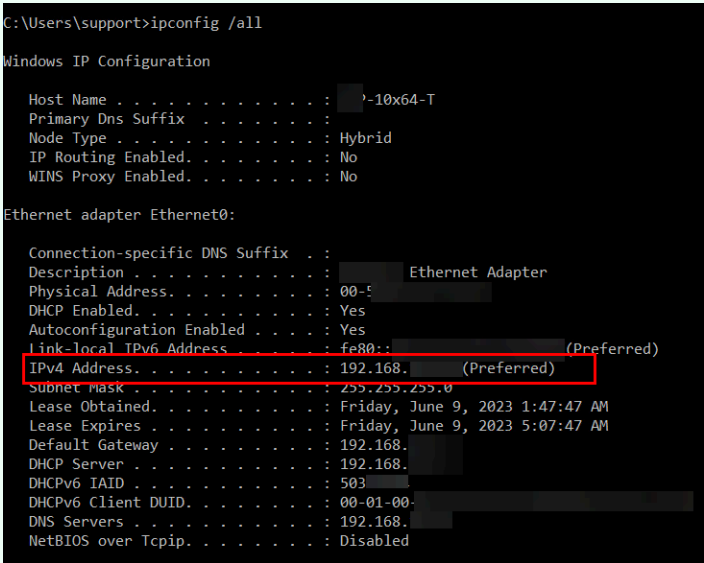
2. Go to the folder **Tier0-Edge/deploy** inside the project.

```
cd Tier0-Edge/deploy
```

3. Edit the following attributes in file **.env.default**.

✓ **tip**

Before editing **.env.default** file, execute **ipconfig** in Git Bash to get the IP of the server.



```
C:\Users\support>ipconfig /all

Windows IP Configuration

Host Name . . . . . : ?-10x64-T
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet0:

Connection-specific DNS Suffix . :
Description . . . . . : Ethernet Adapter
Physical Address. . . . . : 00-50-56-80-00-00
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::... (Preferred)
IPv4 Address. . . . . : 192.168.1.10 (Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Friday, June 9, 2023 1:47:47 AM
Lease Expires . . . . . : Friday, June 9, 2023 5:07:47 AM
Default Gateway . . . . . : 192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 503
DHCPv6 Client DUID. . . . . : 00-01-00-00-00-00-00-00-00-00-00-00-00-00-00-00
DNS Servers . . . . . : 192.168.1.1
NetBIOS over Tcpip. . . . . : Disabled
```

- **OS_PLATFORM_TYPE**: The operating system where Tier0 is installed. Change it to **windows**
- **VOLUMES_PATH**: The storage path for project data.
- **ENTRANCE_DOMAIN/ENTRANCE_PORT**: Tier0 access address. Normally the IP or domain of the server where you install Tier0, and make sure the port is not occupied.

```
vi .env.default
```

① **note**

.env.default file is hidden, directly enter the command to open it.


```

MINGW64/d/supOS-CE
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=windows

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=2
OS_NAME=supos
VOLUMES_PATH=/c/Users/Free/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work, )
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192
ENTRANCE_PORT=8

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US

# Operating system version
OS_VERSION=V1.00.00.05-C

# MQTT TCP port
OS_MQTT_TCP_PORT=1883

# MQTT WebSocket TLS port
OS_MQTT_WEBSOCKET_TLS_PORT=8084

# Login path for the system
OS_LOGIN_PATH=/supos-login

# Enable or disable authentication (true/false)
OS_AUTH_ENABLE=true

# Type of large language model used
OS_LLM_TYPE=ollama

# Model and Key for oepnai (Enter your api key, can be empty)
OPENAI_API_MODEL=gpt-4o
OPENAI_API_KEY=

# Below are the environment variables required for using the built-in supos MCP with the MCP Client (if applicable, can be empty)
# Langsmith API key
LANGSMITH_API_KEY=
# Agent service address (default: http://localhost:8123)
AGENT_DEPLOYMENT_URL=http://mcpclient:8123
# open API key for supos, used for external access to the system
SUPOS_API_KEY=4174348a-9222-4e81-b33e-5d72d2fd7f1e
# The address of the MQTT server of supos that needs to be accessed
SUPOS_MQTT_URL=tcp://emqx:1883/mqtt

#Variables from older versions, will be deleted for next iteration, please do not modify
MULTIPLE_TOPIC=false

#-----OAUTH CONFIG-----
OAUTH_REALM=supos
.env [unix] (10:22 20/05/2025)
".env" [unix] 69L, 1988B

```

4. Save the file and start installing Tier0.

```

:wq!
bash bin/install.sh

```

5. Select the default configurations and wait for the installation to complete.

```
MINGW64:/d/supOS-CE
Free@DESKTOP-JK7K19R MINGW64 /d/supOS-CE (main)
$ bash bin/startup.sh
Choose VOLUMES_PATH: (Press Enter for default: [/c/Users/Free/volumes/supos/data])
Choose IP address for ENTRANCE_DOMAIN (Press Enter for default: [192.168.1.1]):
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /d/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_amd64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_amd64
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_amd64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
golioth-websocket-datasource/plugin.json
golioth-websocket-datasource/README.md
Info: success to create folder: /c/Users/Free/volumes/supos/data
time="2025-05-20T10:22:57+08:00" level=warning msg="D:\\supOS-CE\\docker-compose-8c16g.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 14/14
  ✓ Network supos_default_network Created 0.0s
  ✓ Container postgresql Healthy 62.0s
  ✓ Container eventflow Started 0.8s
  ✓ Container chat2db Started 0.8s
  ✓ Container konga Started 0.4s
  ✓ Container emqx Started 1.0s
  ✓ Container portainer Started 0.6s
  ✓ Container frontend Started 0.6s
  ✓ Container hasura Started 61.3s
  ✓ Container nodered Started 1.2s
  ✓ Container keycloak Healthy 81.9s
  ✓ Container grafana Started 61.4s
  ✓ Container backend Started 82.1s
  ✓ Container kong Started 82.3s
>> start to init nodered modules ...
added 55 packages in 4s
28 packages are looking for funding
  run 'npm fund' for details
npm warn deprecated node-opcua-client-crawler@2.127.1: true
```

6. Access Tier0 on a browser and log in.

Uninstalling Tier0

1. Access the path **Tier0-Edge/deploy/bin**.
2. Execute the script **uninstall.sh**.

```
bash uninstall.sh
```

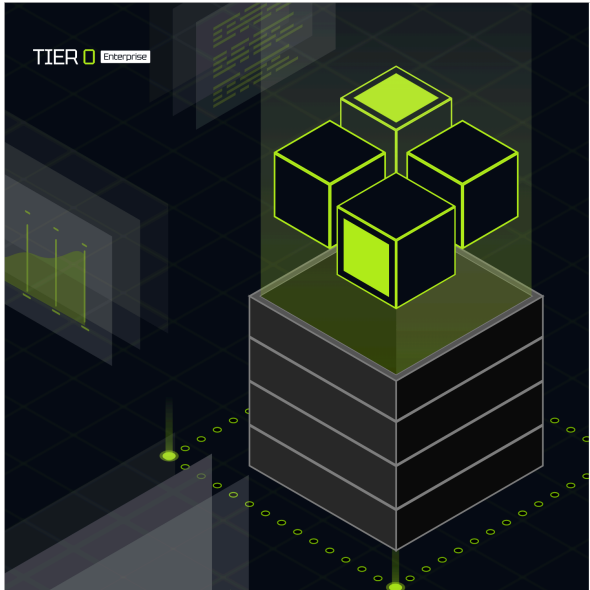
① info

- **clean-all.sh** will wipe out all project data and uninstall Tier0.
- Images will not be deleted.
- If you want to install Tier0 again, make sure you have pulled all images with the newest versions (based on image tags).

Login

1. Access Tier0 with the set domain and port on a browser.
2. On the login page, use your license to activate Tier0.

- **Online Activation:** Enter the license key, and click **Activate Now**.



License Activation

[Online Activation](#) [Offline Activation](#)

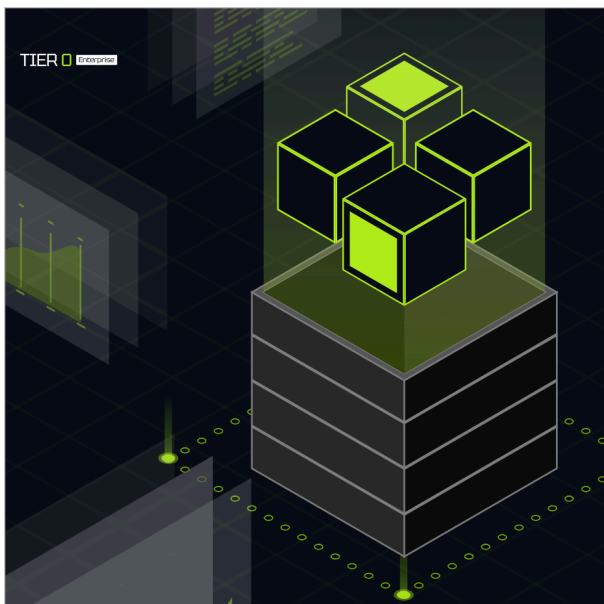
License Key

Activate Now

Need help?

- Contact us at support@tier0.dev
- Visit our [documentation](#) for activation guides

- **Offline Activation:** Send the token to your vendor to get the JSON file, and import the file.



License Activation

[Online Activation](#) [Offline Activation](#)

1 Send the device token to the vendor, then paste the offline Bundle JSON below to import.

Device Token (send to vendor for offline license)

Bundle JSON

Paste the Bundle JSON provided by the vendor

Import Bundle

Need help?

- Contact us at support@tier0.dev
- Visit our [documentation](#) for activation guides

3. Enter the default username and password (tier0/tier0).
4. Click **Sign In**.

How-to Guide

Build Data Models

Tier0 uses UNS (Unified Namespace) to organize data into a clear tree structure. Each path and topic in the model automatically generates an MQTT topic, making it easy to get real-time data.

Additional Parameter Definition

- **Path-Additional Settings**

Item	Description
Extended Attribute	Add extended attributes for the path. Eg. unit.
Template	Select a template to inherit attributes from it. See Building Template .
Generate Template	Click  to add an attribute to the path, at the same time you can select to generate a same template for future use.

- **Topic-Topic Type**

Type	Description
State	Works as an API to receive system state data.
Action	Works as an API to execute CRUD commands.
Metric	Real-time data. For example, equipment temperature.

- **Topic-Attribute Generation Method**

Attribute Generation Method	Description
Custom	Customize attributes.
Template	Inherit attributes from the selected template.
Auto Parsing	Use JSON text to generate attributes with the same structure.

- **Topic-Linked Operation**

Item	Description
Enable History	Enable historical data storage for the topic.

How to Build a Data Model

Based on simple folder-file structure, you can define the data hierarchy to a tree map.

Building Models Manually

① note

factory/equipment/CNC will be used as an example, in which `factory` and `equipment` are paths and `CNC` is a topic.

1. Log in to Tier0, landed on the **Namespace** page.
2. Under **Topic**, click **+ > New Path** to add a path (e.g. `factory`).

New Path

×

Namespace ?

* Name

▼ Additional Settings

* Alias ?

__d45036a7cb0e4c258979

Display Name

Path Description

Extended Attribute

+

Template

Custom

▼

Attribute ?

+

Save

3. Enter the path information, set the alias as needed, and then click **Save**.
4. Select **equipment**, and then click **+** > **New Topic** to add a topic (e.g. CNC) under it.

New Topic
×

Namespace ?

* Name

Topic Type ?
☒ State
☐ Action
☐ Metric

Attribute Generation ?
☒ Custom
☐ Template
☐ Auto Parsing

Method

Attribute ?

Additional Settings

* Alias ?

Display Name

Topic Description

Label ?

Extended Attribute

Optional Behaviors

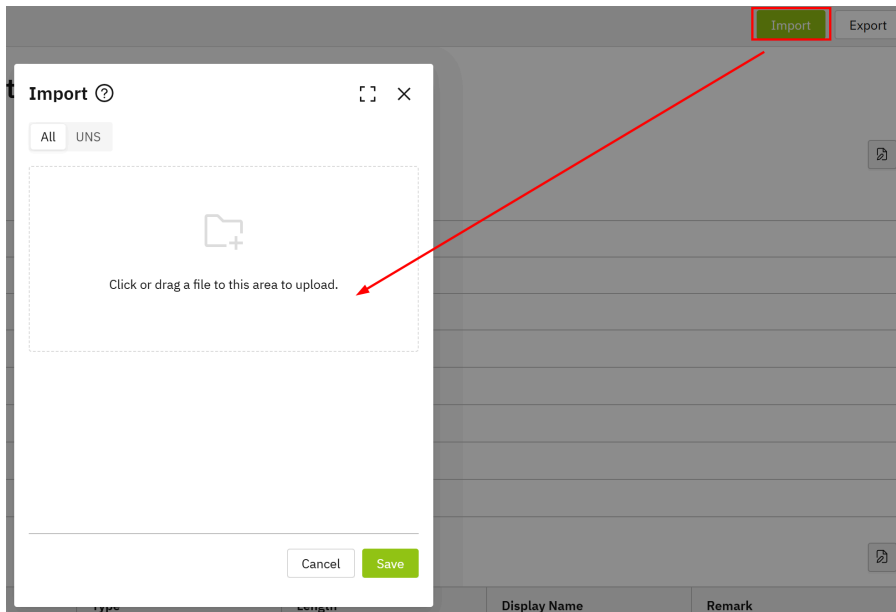
☐ Enable History ?

Save

5. Enter the information of the topic, and then click **Save**.

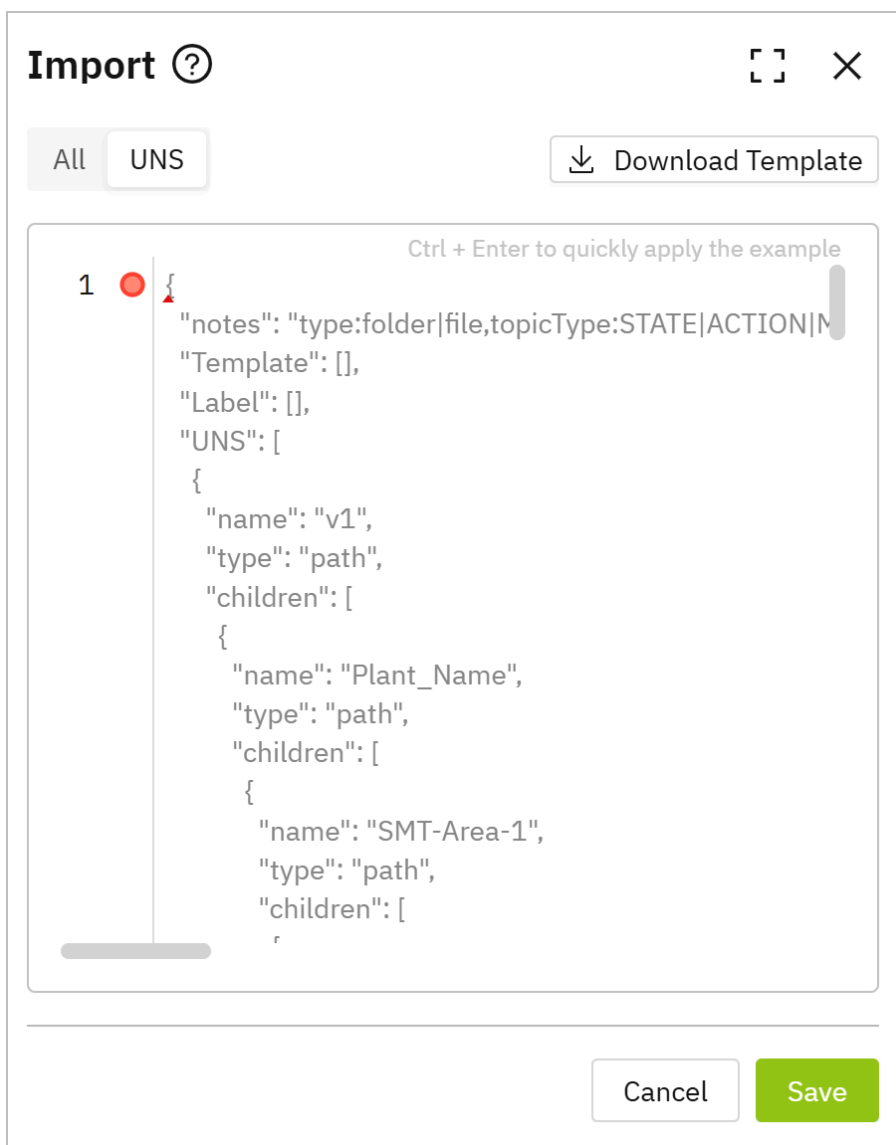
Importing Models

1. Log in to Tier0, landed on the **Namespace** page.
2. Click **Import** at the upper-right corner.



3. Import JSON to create models.

- Click **UNS**, and directly enter JSON.

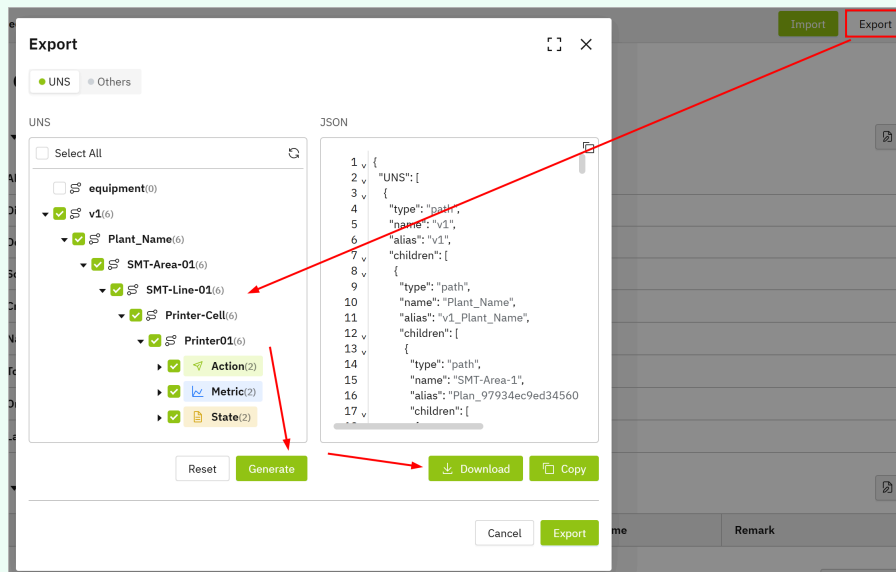


- Click **Download Template** at the **UNS** tab, enter the model content and upload it at the **All** tab.

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION",
  "Label": [{"name": "debug"}, {"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
                              "topicType": "STATE"
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

✓ tip

You can manually add a path and topic, export it and use it as an example for import.



- Click **Save**.

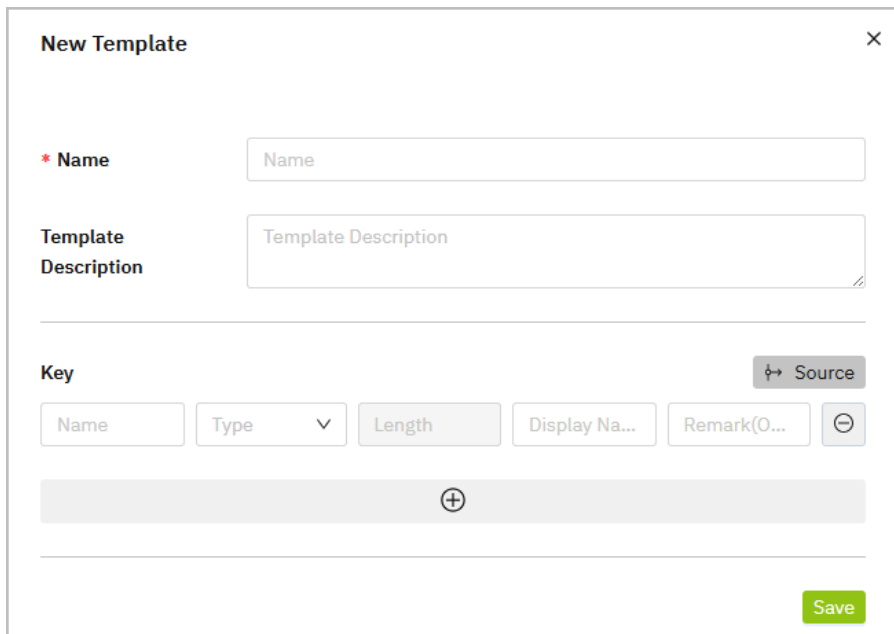
Optional Features

Building Template

1. On the **Namespace** page, click **Template**.
2. Click  , and then enter the information of the template.

✓ tip

Click **Source** to select added models and inherit their attributes.



The 'New Template' dialog box contains the following fields and controls:

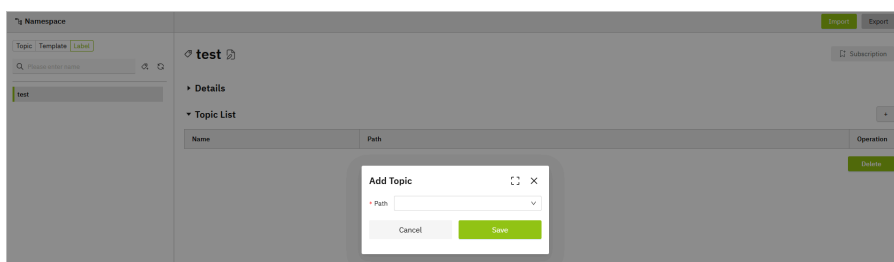
- Name:** A text input field with the placeholder text 'Name'.
- Template Description:** A text area with the placeholder text 'Template Description'.
- Key:** A section containing a 'Source' button (with a key icon) and a row of input fields: 'Name', 'Type' (with a dropdown arrow), 'Length', 'Display Na...', and 'Remark(O...'. There is also a minus icon button.
- Footer:** A 'Save' button.

3. Click **Save**.

Creating Label

Labels are used for categorizing data models.

1. On the **Namespace** page, click **Label**.
2. Click  , and then enter the label name.
3. Click **Save**, and then click  to add topics under the label.



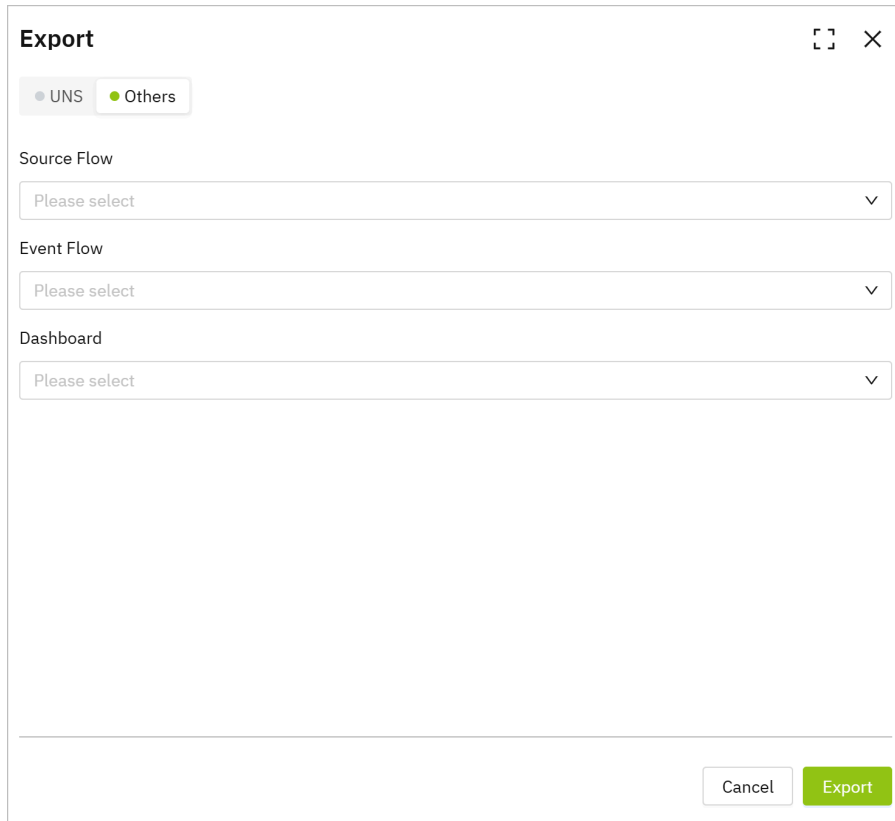
The screenshot shows the 'Namespace' page with a 'test' label selected. An 'Add Topic' dialog box is open, featuring a 'Path' input field and 'Cancel' and 'Save' buttons.

4. Click **Save**.

Exporting Data

Tier0 can export UNS models and other data, such as source flows, event flows, and data dashboards.

1. On the **Namespace** page, click **Export** at the upper-right corner.
2. Click the **Others** tab and select the corresponding data type.

The image shows a modal dialog box titled "Export" with a close button (X) in the top right corner. Inside the dialog, there are two tabs: "UNS" and "Others". The "Others" tab is selected and highlighted with a green dot. Below the tabs, there are three sections, each with a label and a dropdown menu:

- Source Flow**: A dropdown menu with the text "Please select" and a downward arrow.
- Event Flow**: A dropdown menu with the text "Please select" and a downward arrow.
- Dashboard**: A dropdown menu with the text "Please select" and a downward arrow.

At the bottom right of the dialog, there are two buttons: "Cancel" and "Export". The "Export" button is green and highlighted.

3. Click **Export**.

Connect Data Sources

How to Add Data Sources

Once the data model is built under **Namespace**, you need to connect real data to the model for subsequent processing and use.

Adding Data Flow Manually

1. Log in to Tier0, go to **UNS > Source Flow**.

Source Flow						1	Search	New Source Flow
Name	Flow Template	Description	Creation Time	Creator	Operation	2	3	4
factory/equipment/Metric/CNC2 Running	node-red		17:38:12 25/11/2025	supos				
test Draft	node-red		17:12:23 25/11/2025	supos				

Total 2 Items < 1 > 18 /page

Index	Item	Description
1	Search	Search for existing data flows.
	New Source Flow	Add a data flow.
2	Flow List	Displays flow information. Click the flow name to start editing the data flow.
3	Display	Switch flow display between blocks and list.
4	Flow Operations	Copy / Edit / Delete the data flow.

- Click **New Source Flow**, enter a name and click **Save**.
- Click the flow you just added, and drag in nodes according to your data source type (e.g. modbus).
- Drag in an mqtt out node, and finish its configurations.
 - Add the internal broker via emqx:1883 .

Edit mqtt out node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server Port

☒ Connect automatically

☐ Use TLS

Protocol

Client ID

Keep Alive

Session ☒ Use clean session

- Set its subscription topic to the data model in **Namespace**.

For example, the data model structure is `factory/equipment/cnc` , the topic will be `factory/equipment/cnc` .

Edit mqtt out node

Delete Cancel Done

Properties

Server emqx:1883

Topic factory/equipment/cnc

QoS Retain

Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

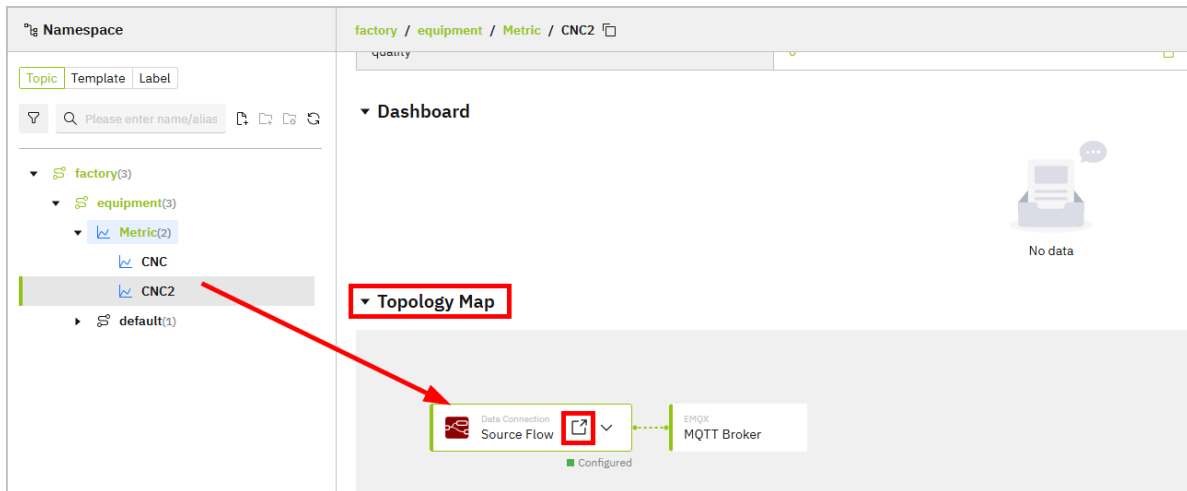
5. Click **Deploy** at the upper-right corner, trigger the flow and the data will be sent to corresponding model under **Namespace**.

Configuring Data Source through Generated Flow

① note

Data flow will only be automatically generated when you select **Mock Data** while creating topics.

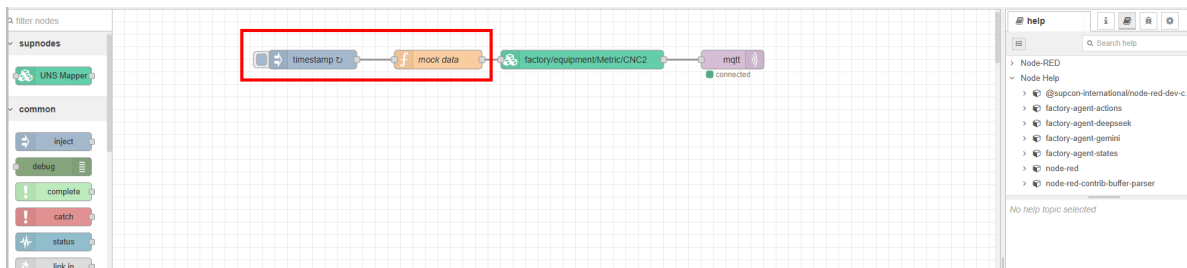
1. Log in to Tier0, landed on the **Namespace** page, and under the **Topic** tab, select a file.
2. Scroll down to **Topology Map**, click the icon on **Source Flow** to redirect to the generated data flow under **Source Flow**.



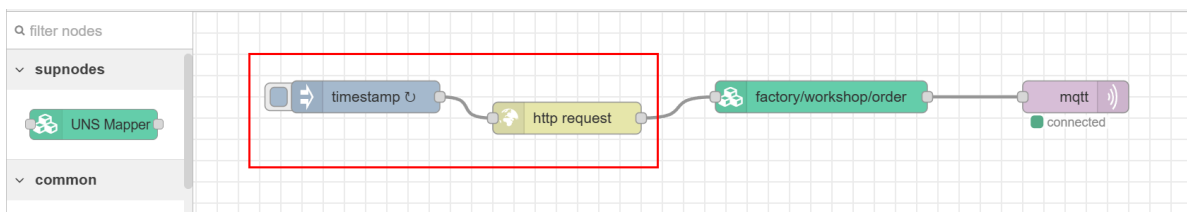
3. Change the data source of the generated flow.

info

Make sure the returned data of the source node is **JSON object**. Currently, Tier0 can only parse this data type.



4. Delete the `mock data` function node and connect the source node to the rest of the flow.



5. Click **Deploy** at the upper-right corner, and then trigger the flow.

6. Go back to **Namespace**, check whether the data is received.

Payload		
Key Name	Value	Latest Update
actualRuntime	1285	11:24:55 16/05/2025
plannedRuntime	1440	11:24:55 16/05/2025

Common Data Sources

Connecting OPC UA

Read Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

Dialog box titled "Edit inject node". It has buttons for "Delete", "Cancel", and "Done". The "Properties" tab is selected. The "Name" field is empty. The "msg. payload" field is set to "milliseconds since epoch". The "msg. topic" field is highlighted with a red box and set to "ns=2;i=2".

3. Drag an **OpCua-Client** node, and then double-click it.
4. Click **+** next to **Endpoint** to enter the information of your OPC UA server.

Dialog box titled "Edit OpCua-Client node > Add new OpCua-Endpoint config node". It has buttons for "Cancel" and "Add". The "Properties" tab is selected. The "Endpoint" field is set to "opc.tcp://192.168.1.104:40". The "SecurityPolicy" field is set to "None". The "SecurityMode" field is set to "None". The "Anonymous" checkbox is checked. The "use credentials" and "user certificate" checkboxes are unchecked.

5. Click **Add**, and set **Action** to **READ**.
6. Leave everything else as default, and then click **Done**.

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint: opc.tcp://192.168.1.100:4840

Action: READ

Certificate: None, use generated self-signed certificate

Local certificate file, fixed path: Path is not used anymore! Example: selfSigned.cer

Local private key file, fixed path: Path is not used anymore! Example: private_key.pem

Use transport settings: ☐

Max ChunkCount: 1

Max MessageSize: 8192

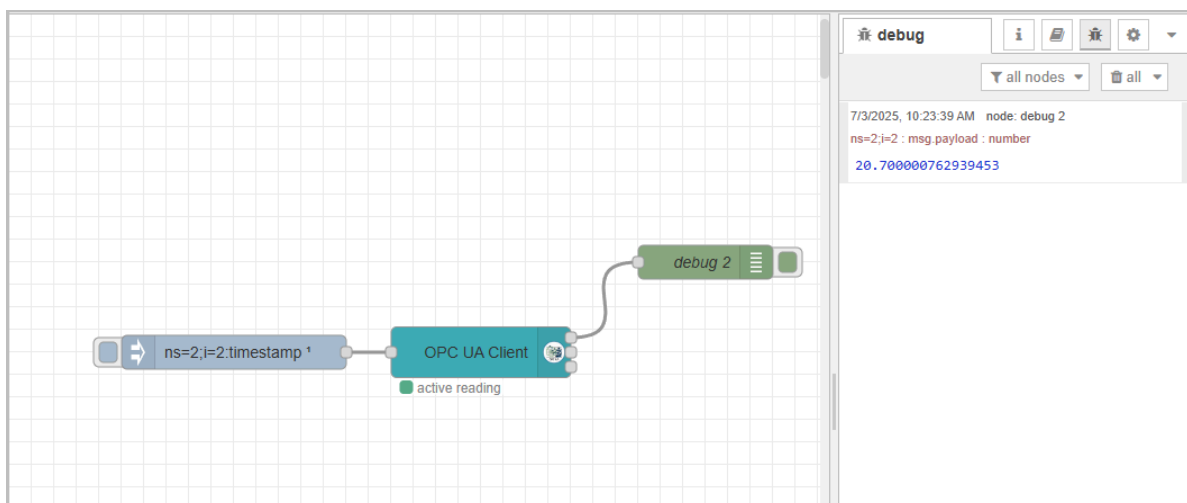
Receive BufferSize: 8192

Send BufferSize: 8192

Set statuscode and timestamp: ☒

Keep session alive: ☒

7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.
8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Subscribing Data from OPC UA Server

1. In **Source Flow**, drag an **inject** node.
2. Double-click the node, and then enter the **NodeId** from your OPC UA server.

- msg.payload:

```
[
  {
    "nodeId": "ns=2;i=1001",
    "datatype": "Int32"
  },
  {
    "nodeId": "ns=2;i=1002",
    "datatype": "Int32"
  }
]
```

- msg.topic: multiple

Dialog box titled "Edit inject node" with buttons "Delete", "Cancel", and "Done". The "Properties" tab is selected. The "Name" field is empty. The "msg.payload" field is set to a JSON array of two objects, each with "nodeId" and "datatype" properties. The "msg.topic" field is set to "multiple". A red rectangle highlights the "msg.payload" and "msg.topic" fields.

3. Drag an **OpCua-Client** node, and then double-click it.

4. Click **+** next to **Endpoint** to enter the information of your OPC UA server.

Dialog box titled "Edit OpCua-Client node > Add new OpCua-Endpoint config node" with buttons "Cancel" and "Add". The "Properties" tab is selected. The "Endpoint" field is set to "opc.tcp://192.168.1.100:4840". The "SecurityPolicy" field is set to "None". The "SecurityMode" field is set to "None". The "Anonymous" checkbox is checked. The "use credentials" and "user certificate" checkboxes are unchecked.

5. Click **Add**, and set **Action** to **SUBSCRIBE**.

6. Set the subscription interval, and leave everything else as default, and then click **Done**.

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint: opc.tcp://192.168.1.104:4840

Action: SUBSCRIBE

Interval: 2 second(s)

Certificate: None, use generated self-signed certificate

Local certificate file, fixed path: Path is not used anymore! Example: selfSigned.cer

Local private key file, fixed path: Path is not used anymore! Example: private_key.pem

Use transport settings: ☐

Max ChunkCount: 1

Max MessageSize: 8192

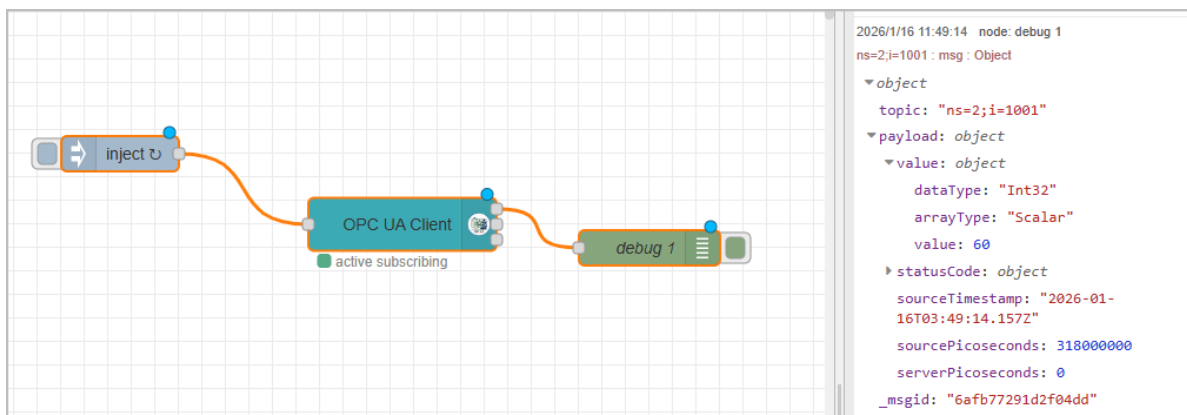
Receive BufferSize: 8192

Send BufferSize: 8192

Set statusCode and timestamp: ☐

7. Connect the **inject** node to the **OpcUa-Client** node, and then connect a debug node at the end.

8. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting OPC DA

1. In **Source Flow**, drag an **inject** node.
2. Drag an **opcda-read** node, and then double-click it.
3. Click **+** next to **Server** to enter the information of your OPC DA server.

Edit opcda-read node > Add new opcda-server config node

Cancel Add

Properties

Name: opcda

Address: 192.168.31.142

Domain: "

User Name: admin

Password:

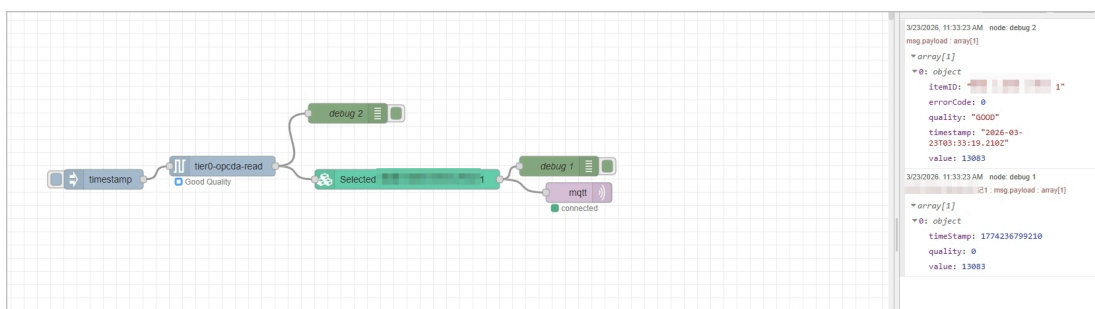
ClsId: 7BC0CC8E-482C-47CA-ABDC-0FE7F9C6E729

Timeout: 5000 ms

Browse Export

Parameter	Description
Address	OPC DA server IP address.
Domain	The domain name of your OPC DA server.
User Name/Password	The username and password you use to log in to the DA server.
ClsId	The application ID of the DA connection service you use. For example, KepServer.

- Click **Browse** to see the connected channels.
- Click **Add**, and then under **Items**, add the items needed.
- Connect the **inject** node to the **opcda-read** node, and then connect a debug node at the end.
- Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting Modbus

1. In **Source Flow**, drag a **Modbus Read** node.
2. Double-click the node, and then click **+** next to **Server** to enter the information of your modbus server.

Edit Modbus-Read node > Add new modbus-client config node

Cancel Add

Properties

Settings Queues Optionals

Name modbus

Type TCP

Host 192.168.1.100

Port 502

TCP Type DEFAULT

Unit-Id 1

Timeout (ms) 1000

Reconnect on timeout ☒

Reconnect timeout (ms) 2000

3. Click **Add**, and then enter the information of the data to be connected.

Edit Modbus-Read node

Delete Cancel Done

Properties

Settings Optionals

Name modbus

Topic Topic

Unit-Id

FC FC 3: Read Holding Registers

Address 0

Quantity 1

Poll Rate 10 second(s)

Delay to activate input ☐

Server modbus

Parameter	Description
Name	The display name of the node (for visual identification only).
Topic	Optional topic string passed to output messages.
Unit-Id	The Modbus slave ID of the target device (typically 1–247).
FC	Function code that specifies the Modbus action, e.g., FC 3 = Read Holding Registers.
Address	The starting register address to read from (usually zero-based).
Quantity	The number of consecutive registers to read.
Poll Rate	How often the node polls the Modbus device (e.g., every 10 seconds).
Delay to activate input	Optional delay before activating the input signal for polling.
Server	Reference to a configured Modbus server (IP, port, protocol, etc.).

4. Click **Done**.
5. Connect a debug node at the end.
6. Click **Deploy** at the upper-right corner, and then click **Modbus Read** node to trigger the flow.

The screenshot shows a Node-RED workspace with a 'modbus' node connected to a 'debug 1' node. The 'modbus' node is orange and has a green status indicator labeled 'active (10 sec.)'. The 'debug 1' node is green. The right sidebar displays the debug console for the 'debug' node, showing four log entries with timestamps and the payload 'array[1]'.

```

7/4/2025, 11:58:23 AM node: debug 1
polling : msg.payload : array[1]
  array[1]
    0: 1

7/4/2025, 11:58:27 AM node: debug 1
polling : msg.payload : array[1]
  array[1]
    0: 1

7/4/2025, 11:58:37 AM node: debug 1
polling : msg.payload : array[1]
  [ 1 ]

7/4/2025, 11:58:47 AM node: debug 1
polling : msg.payload : array[1]
  [ 1 ]

```

Connecting MQTT

1. In **Source Flow**, drag an **mqtt in** node.
2. Double-click the node, and then click  next to **Server**.

Edit mqtt in node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server e.g. localhost Port 1883

☒ Connect automatically

☐ Use TLS

Protocol MQTT V3.1.1

Client ID Leave blank for auto generated

Keep Alive 60

Session ☒ Use clean session

Parameter	Description
Name	Optional name for the MQTT broker configuration (for internal reference).
Server	The hostname or IP address of the MQTT broker (e.g., <code>localhost</code> or <code>broker.hivemq.com</code>).
Port	The port used to connect to the MQTT broker (default is <code>1883</code> for non-TLS, <code>8883</code> for TLS).
Connect automatically	If enabled, Node-RED will try to connect to the broker automatically when the flow starts.
Use TLS	Enable this option if the broker requires a secure TLS/SSL connection.
Protocol	The MQTT protocol version to use for connection (e.g., MQTT v3.1.1 or v5.0).
Client ID	Optional client ID. If blank, a unique one will be auto-generated. Must be unique per client.
Keep Alive	The maximum period (in seconds) between communications with the broker. Helps detect dropped connections.
Use clean session	If enabled, no persistent session is maintained with the broker; otherwise, session state (like subscriptions) is retained.
Username (Security tab)	Username for authenticating with the MQTT broker, if required.
Password (Security tab)	Password corresponding to the username for broker authentication. Stored securely.

3. Click **Add**, and then enter the topic to be subscribed.

4. Click **Done**.

Edit mqtt in node

Delete Cancel Done

Properties

Server: emqx

Action: Subscribe to single topic

Topic: factory/RCS/AGV

QoS: 2

Output: auto-detect (parsed JSON object, string or buff)

Name: Name

5. Connect a debug node at the end.

6. Click **Deploy** at the upper-right corner, and then once the topic receives data, the flow will be triggered.

The screenshot shows the Node-RED interface. On the left, a flow is built with a 'factory/RCS/AGV' MQTT node connected to a 'debug 1' node. The MQTT node is labeled 'connected'. On the right, the debug console is open, showing two log entries from 'node: debug 1'.

7/4/2025, 2:24:45 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

▼ object

- no: 66
- position: "3fcLKwHDAvthuZLEwW8V"
- state: "DqGG213IuzMDT73GnVS3"
- battery: "62.91"

7/4/2025, 2:24:55 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

▶ { no: 60, position: "6CcOvOZfJM8zaCCJA15e", state: "zmP15enJhhK064mK16wV", battery: "63.99" }

Connecting File

info

Before processing files in **Source Flow**, you need to put the file in to the corresponding docker.

```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/  
//copy the file from local to server  
docker cp /root/config.json <docker name or ID>:/app/config.json  
//copy the file from server to docker
```

✓ general docker commands

Command	Description
docker ps	Lists all running Docker containers.
docker ps grep [container]	Filters running containers based on the specified docker information.
docker exec -it [container] /bin/sh	Starts a shell session inside a running container using /bin/sh .
mkdir [directory]	Creates a new directory with the specified name inside a container.

Connecting a Text File

1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

Edit read file node

Delete

Cancel

Done

⚙ Properties

📁 Filename

▼ path /file-test.txt

🔗 Output

a single utf8 string ▼

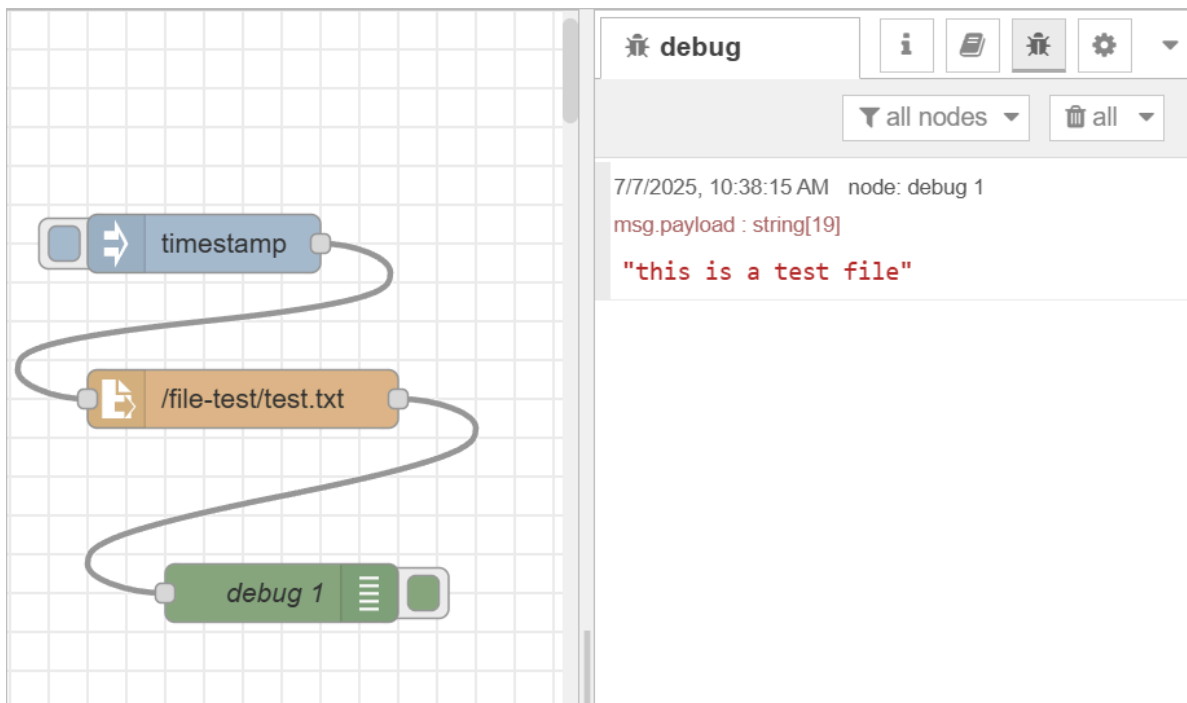
📄 Encoding

default ▼

🏷 Name

Tip: The filename should be an absolute path, otherwise it will be relative to the working directory of the Node-RED process.

4. Click **Done**, and then connect a debug node at the end.
5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.

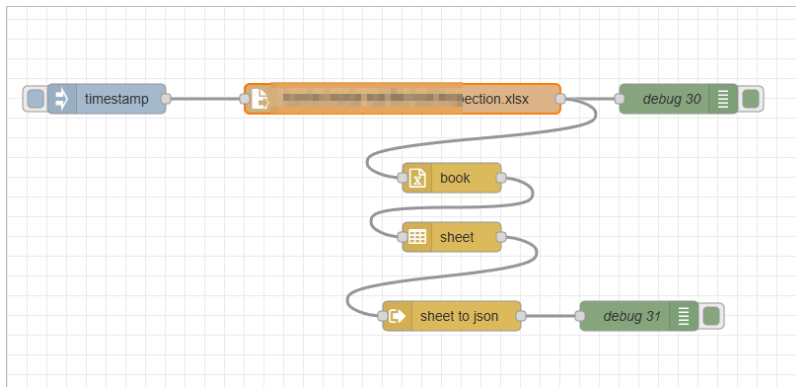


Connecting an Excel File

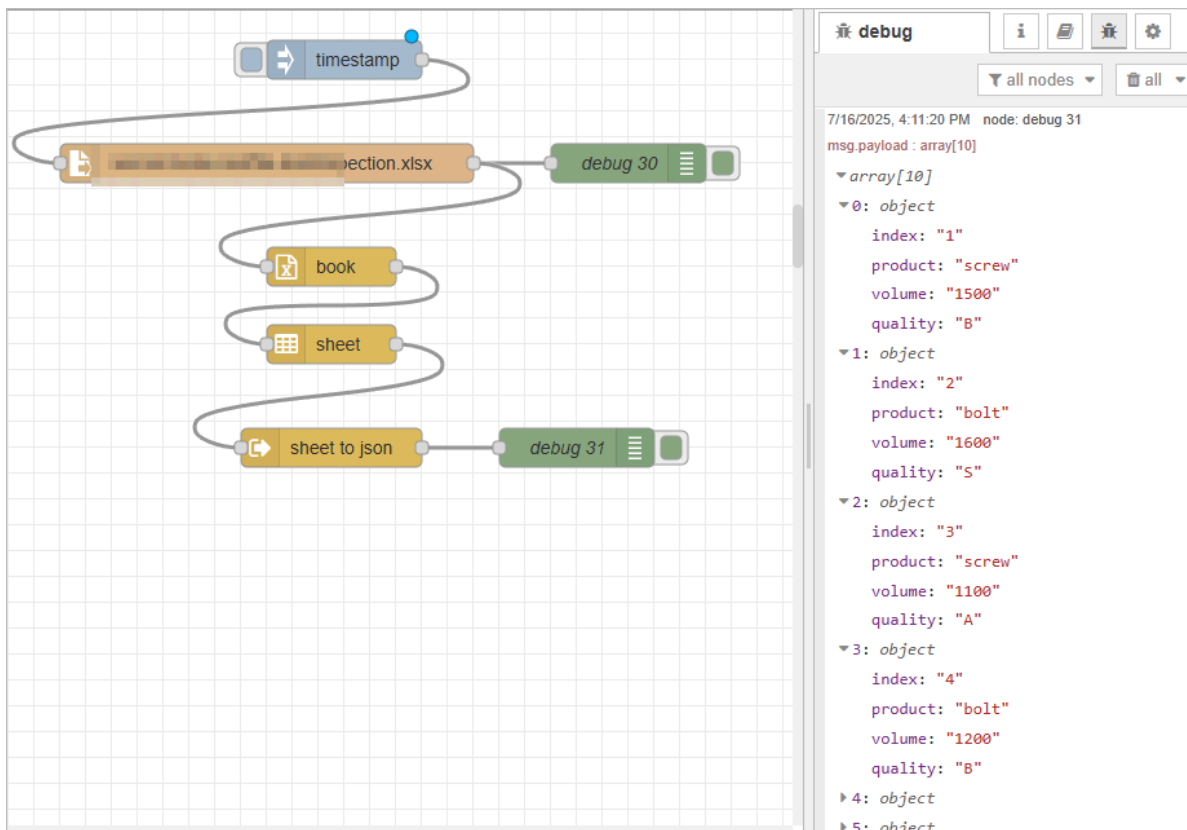
1. In **Source Flow**, drag an **inject** node.
2. Drag a **read file** node, and then double-click it.
3. Enter the file path and output configuration.

The screenshot shows the 'Edit read file node' dialog box. It has a title bar 'Edit read file node' and buttons for 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a gear icon and three sub-sections: 'Filename' with a dropdown menu showing 'path /...ection.xlsx', 'Output' with a dropdown menu showing 'a single Buffer object', and 'Name' with an empty text input field. At the bottom, there is a yellow tip box that reads: 'Tip: The filename should be an absolute path, otherwise it will be relative to the working directory of the Node-RED process.'

4. Click **Done**, and then connect a **book**, a **sheet** and a **sheet to json** node by order to convert table data to json.



5. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Connecting RestAPI

1. In **Source Flow**, drag an **inject** node.
2. Drag an **http request** node, and then double-click it.
3. Enter the API information.

✓ tip

Set the **Return to a parsed JSON object** for Tier0 namespace to parse.

Edit http request node

Delete Cancel Done

Properties

Method GET

URL http://

Payload Ignore

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a parsed JSON object

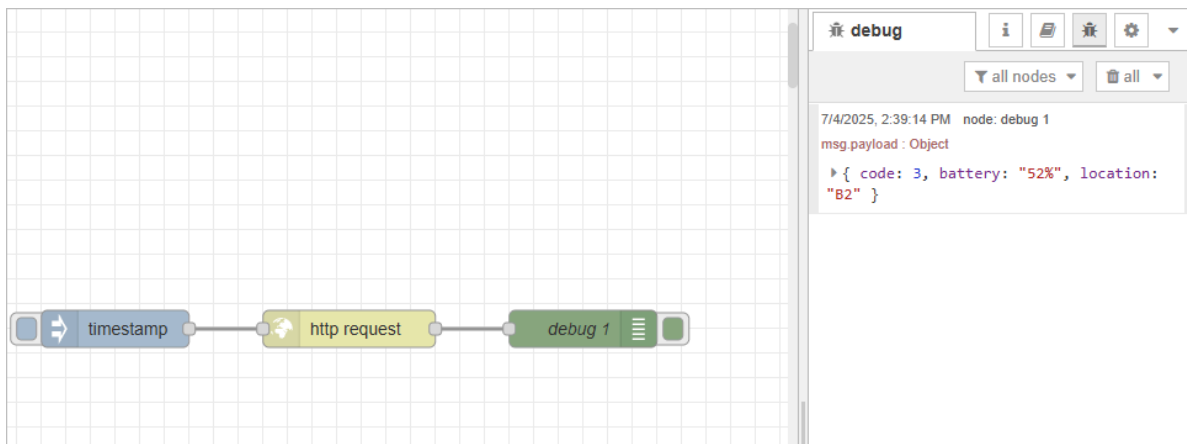
Tip: If the JSON parse fails the fetched string is returned as-is.

Headers

+ add

5. Click **Done**, and then connect a debug node at the end.

6. Click **Deploy** at the upper-right corner, and then click **inject** node to trigger the flow.



Process Data

1. Log in to Tier0, and then select **UNS > Event Flow**.

Event Flow					Please enter name or description		Search	+ New Event Flow
Name	Flow Template	Description	Creation Time	Creator	Operation			
test	Draft	event-flow	17:48:15 25/11/2025	supos				

Total 1 Items < 1 > 19 /page

Index	Item	Description
1	Search	Search for existing data flows.
	New Event Flow	Add a data flow.
2	Flow List	Displays flows information. Click the flow name to start editing the data flow.
3	Display	Switch flow display between blocks and list.
4	Flow Operations	Copy / Edit / Delete the data flow.

- Click **New Event Flow**, enter a name and click **Save**.
- Click the flow you just added, and build data flows to analyze your data.

Common Data Processing Methods

Combining Multiple Sources

Example

There are 3 data sources connected to Tier0, and we need to combine them together for the next step.

```
{
  "workshop": {
    "furnace1": {
      "temp": 85
    },
    "furnace2": {
      "humidity": 64
    },
    "furnace3": {
      "pressure": 80
    }
  }
}
```

What Node to be Used?

In this example, we use **join** node to combine previous messages.

The screenshot shows the 'Edit join node' configuration window. At the top, there are three buttons: 'Delete', 'Cancel', and 'Done'. Below these is a 'Properties' section with a gear icon and three sub-panels (Properties, Code, Visual). The 'Properties' panel contains the following settings:

- Name:** A text input field containing 'Name'.
- Mode:** A dropdown menu set to 'manual'.
- Combine each:** A dropdown menu set to 'msg.payload'.
- to create:** A dropdown menu set to 'a merged Object'.
- ☐ Use existing msg.parts property

Below the properties section, there is a section titled 'Send the message:' with three options:

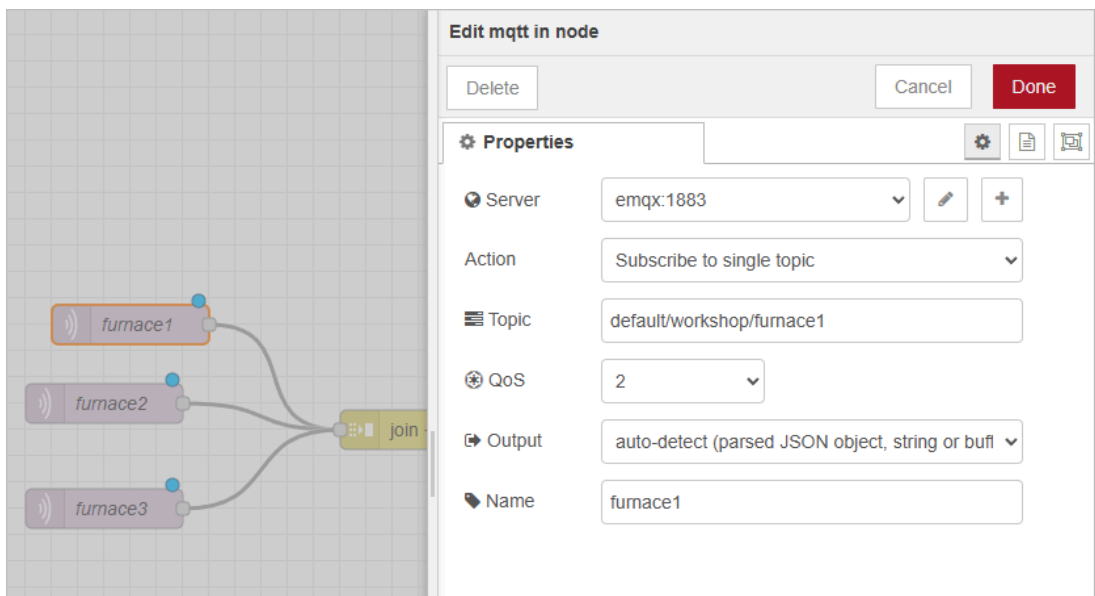
- After a number of message parts: A text input field containing '3', followed by a checked checkbox and the text 'and every subsequent message.'
- After a timeout following the first message: A text input field containing 'seconds'.
- After a message with the `msg.complete` property set

Parameter	Description
Mode	Join mode. • manual: You configure how to merge messages yourself. • auto: Automatically reassemble sequences split by a split node, based on <code>msg.parts</code>. • reduce sequence – Runs a JavaScript reduce function across the sequence to produce a single result.
Combine each	Message part you want to combine. Commonly <code>msg.payload</code>, but can be others (e.g. <code>msg.data.value</code>).
to create	Result type: • a merged Object: Merge all objects into a single object (later keys overwrite same-name keys). • a key/value Object: Use a key property (default <code>msg.topic</code>) as the key, and the Combine each property as the value. • an Array: Append each value to an array in order. • a String / a Buffer: Concatenate text or binary data (String can use a separator; Buffer is joined as binary).
Use existing <code>msg.parts</code> property	When selected, the node uses the <code>msg.parts</code> from previous nodes (usually created by a split node) to determine grouping and when to send the joined output. Fields like <code>msg.parts.count</code> and <code>msg.parts.index</code> are used.
Send the message → After a number of message parts	Outputs the joined result after receiving the set number of fields.
Send the message → and every subsequent message	When selected, after the initial output, the node outputs an updated result upon new messages arrival.
Send the message → After a timeout following the first message	Optional. Starts a timer when the first message arrives; once the timeout (seconds) is reached, the node outputs whatever has been collected.

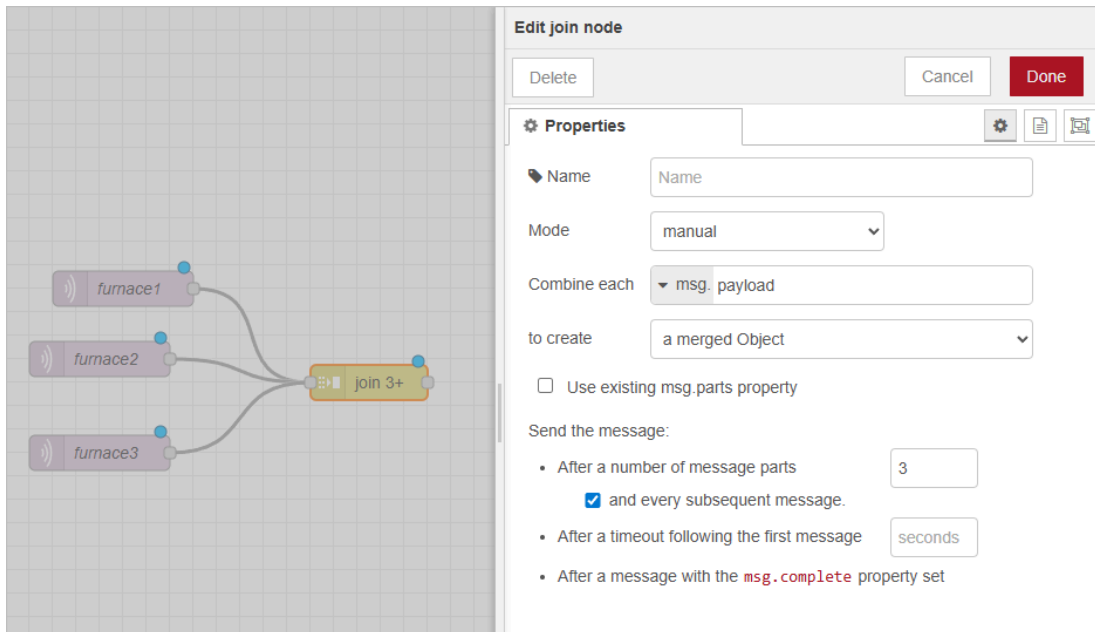
Parameter	Description
Send the message → After a message with the <code>msg.complete</code> property set	Outputs the joined result immediately when a message arrives with <code>msg.complete = true</code> .
(Advanced) String/Buffer separator	For String output, an optional separator can be set. For Buffer output, data is concatenated directly with no separator.

How to Combine Sources?

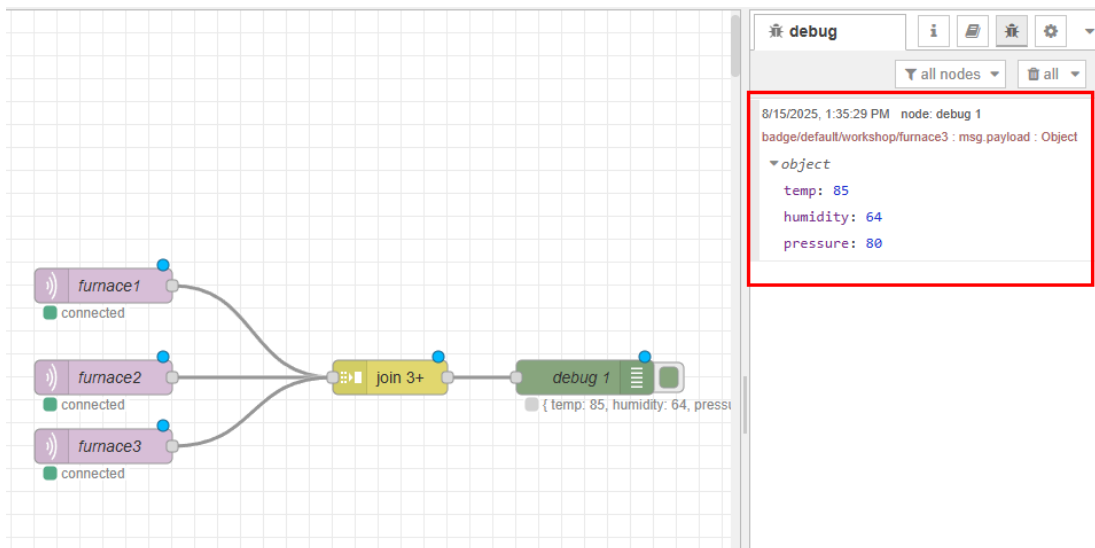
1. In **Event Flow**, drag in 3 **mqtt in** nodes, and subscribe to corresponding topics relatively.



2. Connect a **join** node to all 3 nodes, and merge them into an object.



3. Connect a **debug** node, and trigger the flow to check the results.



Filtering Data

Example

The **furnace** data source contains 3 fields, and we want to route messages based on the temperature threshold:

- If **temp** > 85 → go to **Output 1**
- If **temp** ≤ 85 → go to **Output 2**

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
```

What Node to be Used?

In this example, we use the **switch** node to route messages according to conditions.

Edit switch node

Delete Cancel Done

Properties

Name

Property

→ 1 ✕

value rules

- ==
- !=
- <
- <=
- >
- >=
- has key
- is between
- contains
- matches regex
- is true
- is false
- is null
- is not null
- is of type
- is empty
- is not empty

sequence rules

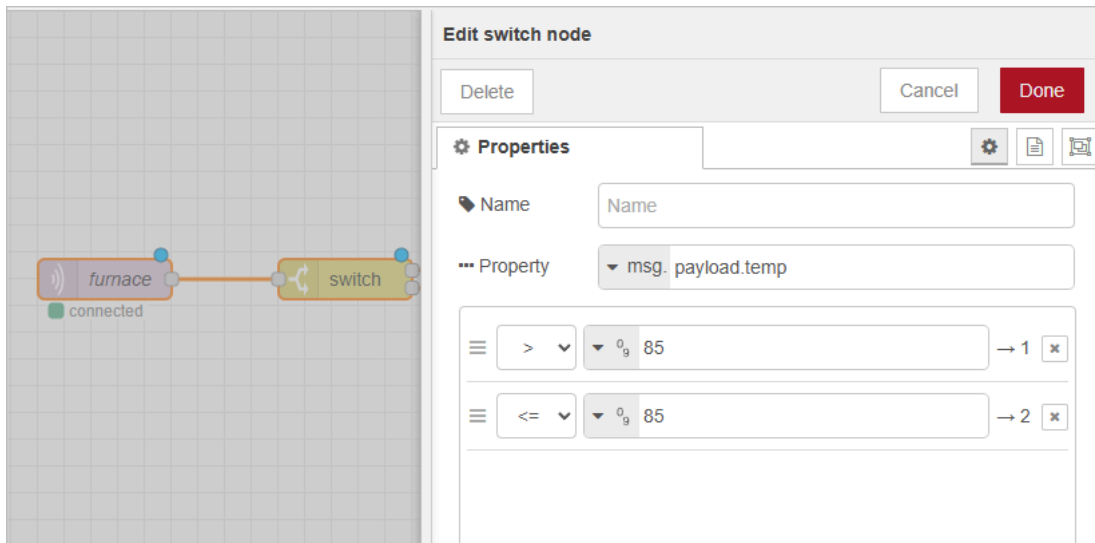
- head

☐ recreate message sequences

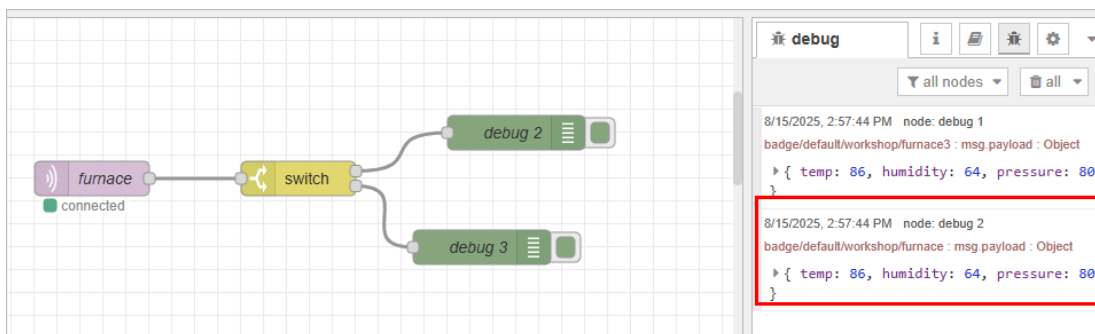
Parameter	Description
Property	The property path to evaluate. Commonly <code>msg.payload</code> or a deeper property, such as <code>msg.payload.temp</code> in this example.
Rules	A set of evaluation rules, each corresponding to one output port.
Type	The data type for comparison: number, string, boolean, JSONata, etc. <i>info</i> Ensures numeric comparisons are not treated as string comparisons.
Value	The value to compare against can come from: a constant value (e.g. 85), <code>msg.</code> fields (e.g. <code>msg.threshold</code>), <code>flow.</code> / <code>global.</code> context, or a JSONata expression.
checking all rules	Evaluates all rules in order. If a message matches multiple rules, it will be sent to multiple outputs.
Stop after first match	Stops checking once the first rule matches, so the message is sent to only one output.
recreate message sequences	When enabled, preserves and rebuilds message sequence metadata (<code>msg.parts</code>)—usually created by a split node—so that the sequence is maintained even after branching.

How to Filter Data?

1. In **Event Flow**, add an **mqtt in** node to get the data from Tier0.
2. Drag in a **switch** node and configure its properties.



3. Connect a **debug** node to each output of the **switch** node.
4. Trigger the flow and check the results.



Changing Data

Example

We receive a message and need to rename fields, calculate new values, and remove unused fields for cleaner downstream processing.

Input message

```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

Target transformation

- Move furnace.temp_c to furnace.temperature.celsius
- Add furnace.temperature.fahrenheit (converted from Celsius)
- Remove pressure
- Change the site from A1 to B1

Output message (example)

```
{
  "furnace":{
    "humidity":64,
    "temperature":{
      "celcius":30.5,
      "fahrenheit":86.9
    }
  },
  "site":"B1"
}
```

What Node to be Used?

In this case, we use the **change** node to perform operations like **Set**, **Change**, **Delete** and **Move** on message properties.

Edit change node

Delete
Cancel
Done

Properties

Name
Name

Rules

Set
▼ msg. payload

to the value
▼ a_z

Change
▼ msg. payload

Search for
▼ a_z

Replace with
▼ a_z

Delete
▼ msg. payload

Move
▼ msg. payload

to
▼ msg.

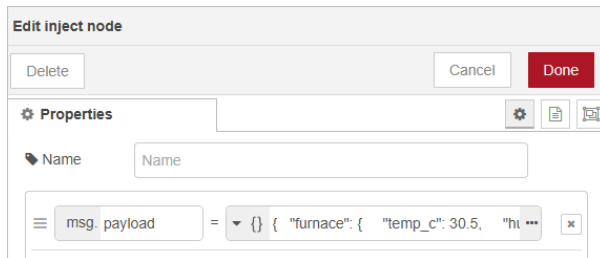
+ add

Parameter	Description
Rules	Each rule is executed in order, top to bottom. Multiple rules can be added to perform complex transformations.
Rule Type	• Set: Set or create a property value. • Change: Find and replace. • Delete: Remove a property. • Move: Move a property from one path to another.
Property/Source/Target	Defines which property to operate on, such as <code>msg.payload</code> or <code>msg.payload.furnace.temp_c</code> . Can also reference <code>flow.</code> or <code>global.</code> context variables.
Target Value	• Set: The value to assign. • Change: The value to find and replace.

How to Use the Change Node?

1. In **Event Flow**, drag in an **inject** node, and add the following json as the input.

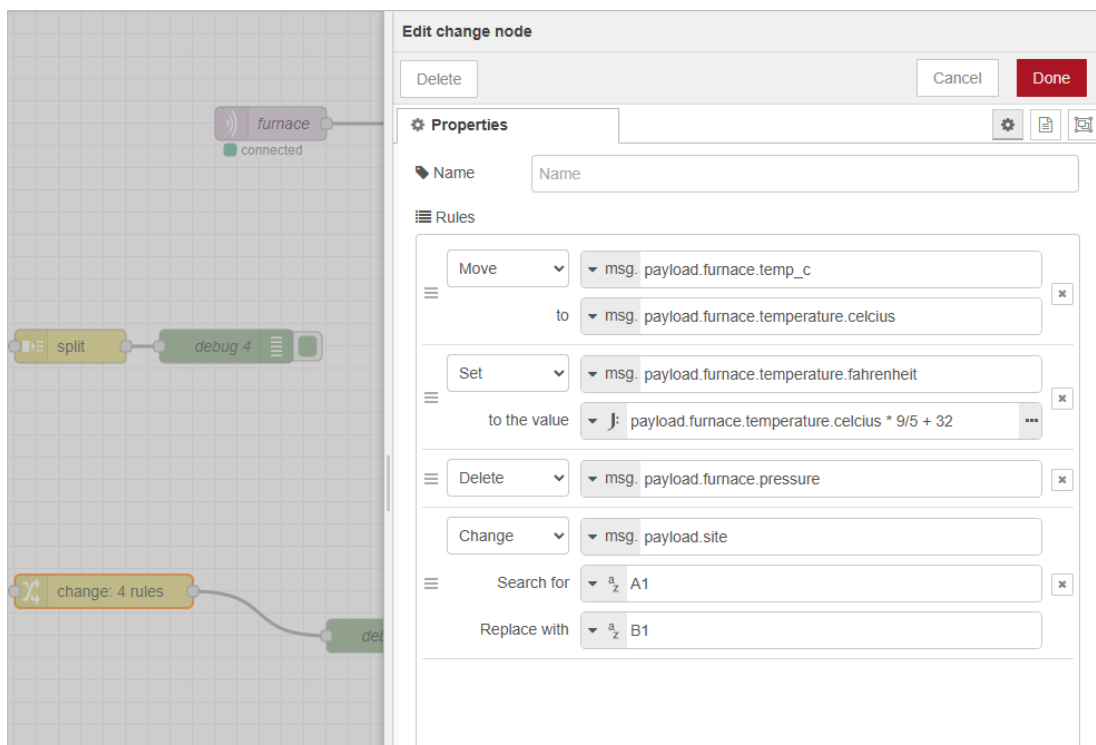
```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```



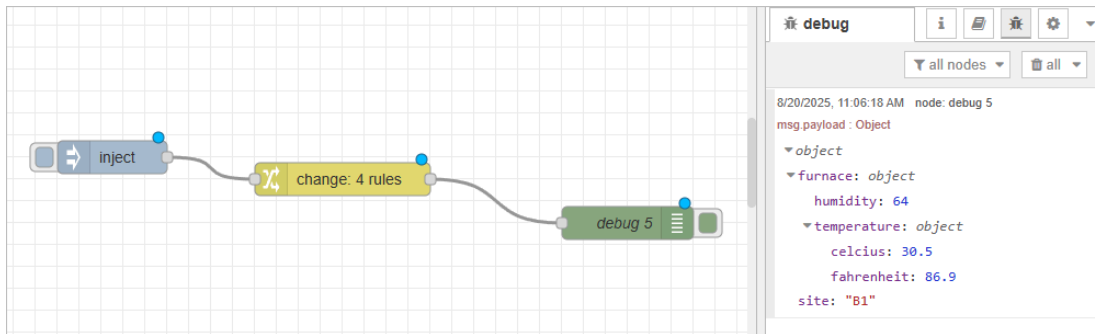
2. Connect it to a **change** node.

3. Open the **change** node configuration panel and add rules.

- Move msg.payload.furnace.temp_c → msg.payload.furnace.temperature.celcius
- Set msg.payload.furnace.temperature.fahrenheit → payload.furnace.temperature.celcius * 9/5 + 32
- Delete msg.payload.pressure
- Change value of msg.payload.site from A1 → B1



4. Connect the **change** node to a **debug** node to check the transformed output.



Spliting Data

Example

We have a single message containing multiple furnace readings, and we need to split them into individual messages for further processing.

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
```

What Node to be Used?

In this example, we use the **split** node to break one message into multiple smaller messages.

Edit split node

Delete

Cancel

Done

Properties



 Name

Name

►► Split the

msg. payload

String / Buffer

Split using

▼ a-z \n

☐ Handle as a stream of messages

Array

Split using

Fixed length of 1

Object

Send a message for each key/value pair

☐ Copy key to

msg.

Parameter	Description
Name	Optional label for the node to help identify it in the flow.
Split the	The property of the message to split, usually <code>msg.payload</code> , but can be any property such as <code>msg.payload.data</code> .
String / Buffer - Split using	For string or buffer payloads, specifies how to split: • string: Use a specific string or regular expression pattern to split the message. • buffer: Defines chunk size by which the message is split. • Fixed length of: Split the string according to the set length. • Leave empty to return the tring unsplit.
Handle as a stream of messages	When checked, the node will process incoming data as a continuous stream, emitting split messages as data arrives, rather than waiting for the full payload.
Array - Split using	Fixed length of N : Splits the array into multiple messages with N elements each.
Object - Send a message for each key/value pair	The value of each key/value pair will be returned as payload.
Copy key to <code>msg.payload</code>	If checked, the original key will be returned as <code>msg.payload</code> of the split messages.

How to Split Messages?

1. In **Event Flow**, drag in an **inject** node, and use a **function** node to transmit the object.

Edit function node

Delete Cancel Done

Properties

Name function 2

Setup On Start **On Message** On Stop

```

1 msg.payload = {
2   "furnace": {
3     "temp": 86,
4     "humidity": 64,
5     "pressure": 80
6   }
7 }
8 return msg;

```

2. Connect it to 2 **split** nodes in order, and configure both properties as the same on the **Object** part.

Edit split node

Delete Cancel Done

Properties

Name Name

Split the msg. payload

String / Buffer

Split using a_z 0

☐ Handle as a stream of messages

Array

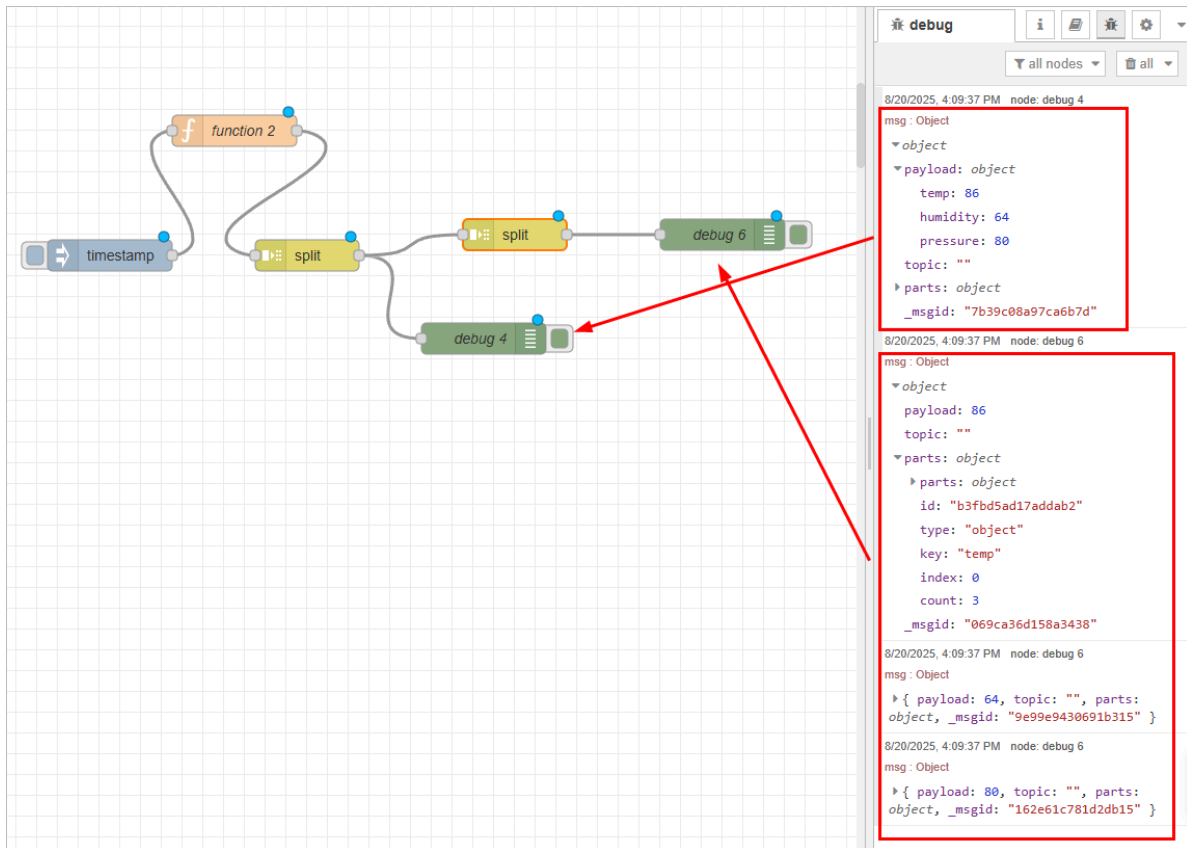
Split using Fixed length of 2

Object

Send a message for each key/value pair

☐ Copy key to msg.

3. Connect the **split** node to a **debug** node to inspect each individual output message.

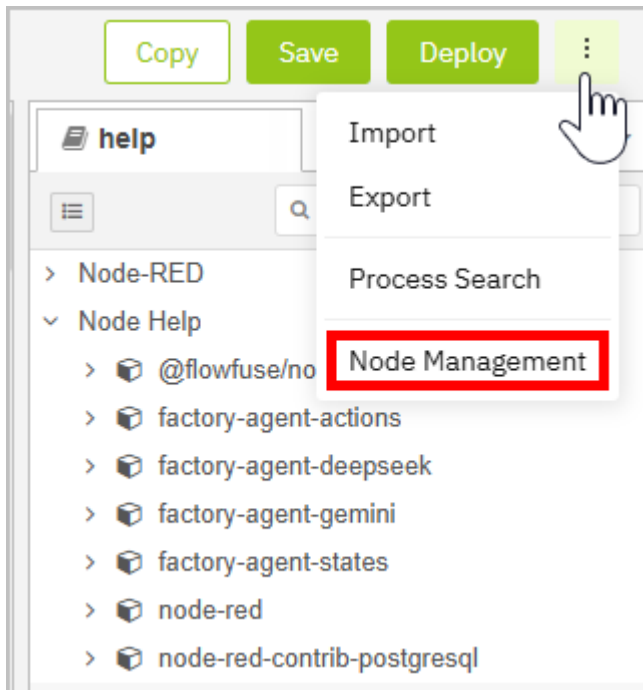


Visualize Data

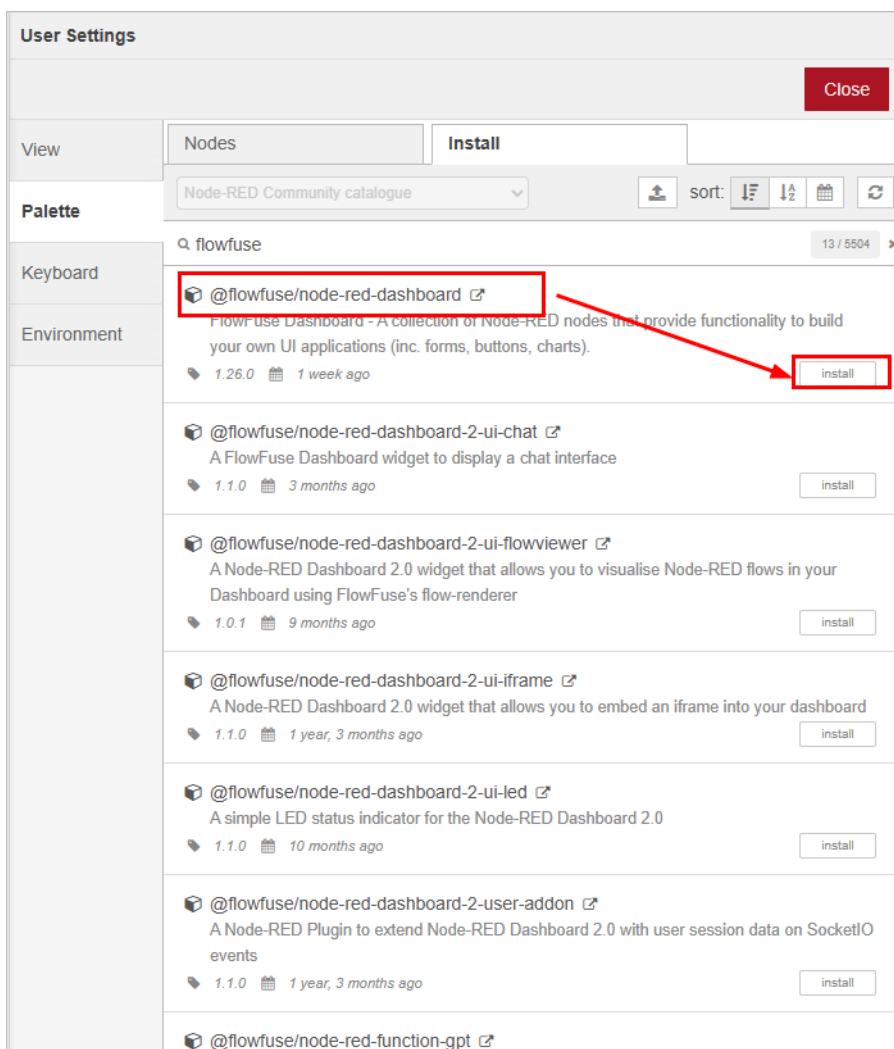
How to Build a Dashboard on Event Flow

Data collected to, processed on Tier0, can be visualized in charts, tables, texts in custom styles and other formats on Tier0 as well.

1. Log in to Tier0, and select **UNS > Event Flow**.
2. Click **New Event Flow** to add a flow.
3. Click the flow name to start editing.
4. Click  at the upper-right corner, and then click **Node Management**.



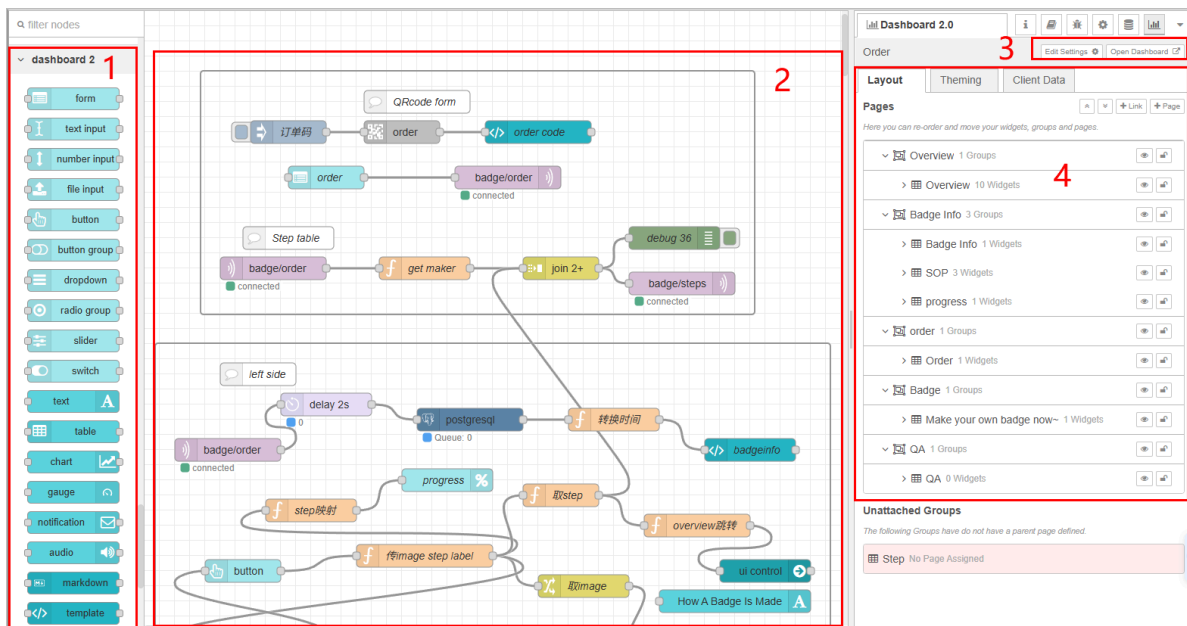
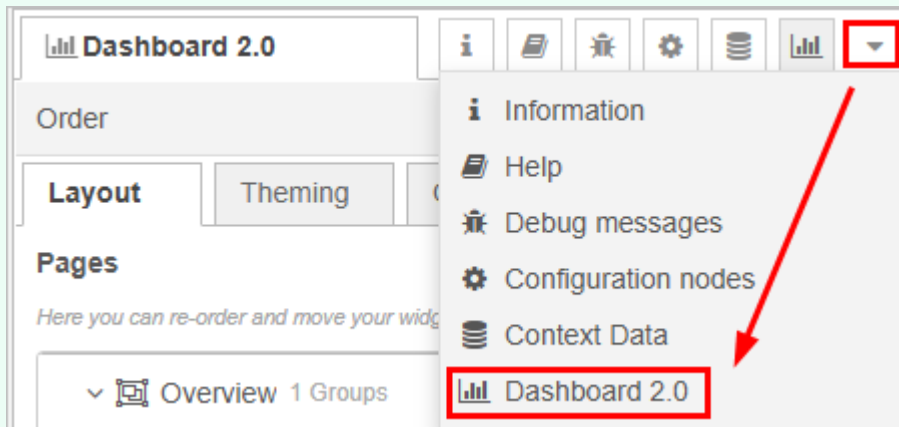
5. Under **Install**, search for and install `@flowfuse/node-red-dashboard`.



6. Use nodes under flowfuse to build a dashboard.

✓ tip

Click  at the upper-right corner, and then click **Dashboard2.0** to adjust the layout.



Index	Item	Description
1	Node	Node of Dashboard 2.0, including gauge, form, progress bar and more.
2	Canvas	Where you drag in nodes to build a dashboard.
3	Operations	Edit the <code>ui-base</code> node and view the dashboard.
4	Layout	Displays all nodes in the corresponding hierarchical layout.

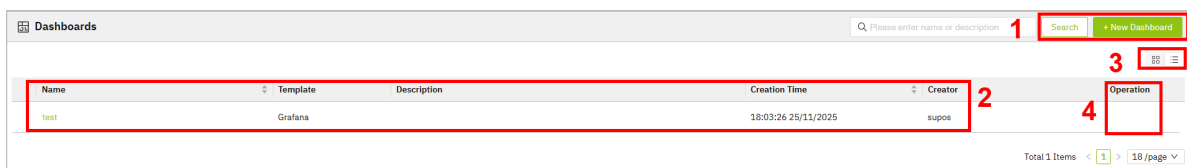
info

For details on how to use each node, see [Flowfuse Node Docs](#).

Display Data on Dashboards

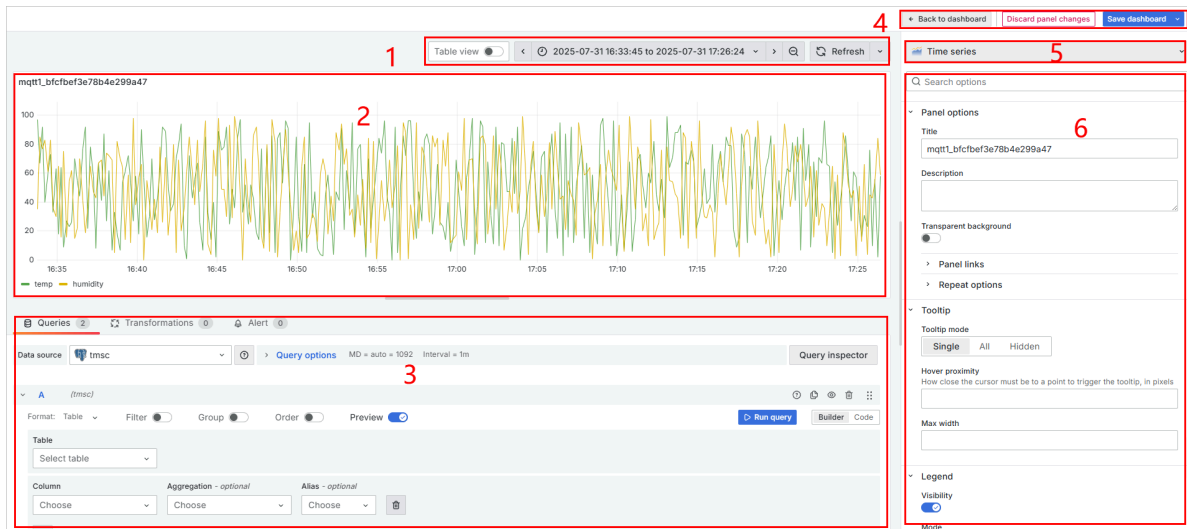
Data collected to, processed on Tier0, can be visualized in charts, tables, heat maps and other formats on a dashboard to provide a comprehensive insight.

1. Log in to Tier0, and select **UNS > Dashboards**.



Index	Item	Description
1	Search	Search for existing dashboards.
	New Dashboard	Add a dashboard.
2	Dashboard List	Displays dashboard information. Click the name to start editing.
3	Display	Switch dashboard display between blocks and list.
4	Operations	Copy / Edit / Delete the dashboard.

2. Click **New Dashboard** to add a dashboard.
3. Click the dashboard name to start editing.
4. Click **Add Visualization**, and then select a data source for the dashboard in the pop-up window.
 - o The default **tmisc** is the internal database where the data collected on **Namespace** is stored.
 - o Click **Configure a new data source** at the lower-right corner to configure other data sources. e.g. MQTT.



Index	Item	Description
1	Tools	Switch between chart and table views.
	Time Range	Select the time range where the data is displayed.
	Refresh	Refresh the element.
2	Display Area	Display the current data source via the selected element.
3	Data Source Configuration	Change data source, and you can either select columns from the database or directly use SQL query to handle complex data.
4	General Operations	Go back to dashboard list/discard the changes on the current panel/save the dashboard.
5	Element	Select different elements for displaying the data.
6	Element Configuration	Configurations of the current element.

Obtain Data from Tier0

Data can be collected to Tier0 through various protocols, it can also be obtained through MQTT subscription, or even directly from Tier0 database.

Obtaining Data through MQTT Subscription

info

This method is only suitable for third parties who have experience in MQTT protocol.

1. Log in to Tier0, and then select **UNS > Event Flow**.
2. Click **New Event Flow**, enter a name and click **Save**.
3. Click the flow you just added, drag an `mqtt in` node, and enter the configurations.
 - Tier0 MQTT broker address: `emqx:1883`
 - Topic: The topic to obtain data from, for example, `factory/workshop/order`.

The screenshot shows the 'Edit mqtt in node' configuration window. It has a title bar 'Edit mqtt in node' and three buttons: 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a settings icon, a document icon, and a preview icon. The properties are: 'Server' (dropdown menu with 'emqx' selected), 'Action' (dropdown menu with 'Subscribe to single topic' selected), 'Topic' (text input field with 'factory/workshop/order'), 'QoS' (dropdown menu with '2' selected), 'Output' (dropdown menu with 'auto-detect (parsed JSON object, string or buf)' selected), and 'Name' (text input field with 'Name').

4. Process the data obtained, or directly use an `mqtt out` node to transmit the data to the third party MQTT.

The screenshot shows the 'Edit mqtt out node' configuration window. It has a title bar 'Edit mqtt out node' and three buttons: 'Delete', 'Cancel', and 'Done'. Below the buttons is a 'Properties' section with a settings icon, a document icon, and a preview icon. The properties are: 'Server' (dropdown menu with 'Your MQTT' selected), 'Topic' (text input field with 'your/topic'), 'QoS' (dropdown menu), 'Retain' (checkbox), and 'Name' (text input field with 'Name'). A red rectangle highlights the 'Server' and 'Topic' fields. At the bottom, there is a yellow tip box that says: 'Tip: Leave topic, qos or retain blank if you want to set them via msg properties.'

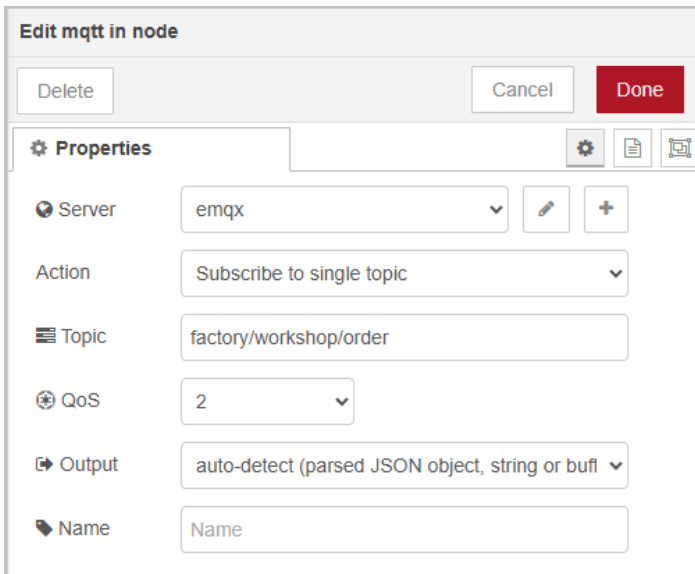
5. Deploy the flow, and data will be pushed to the third party when the topic data changes.

Obtaining Data through API

info

This method is suitable for third parties who do not support MQTT and need event-driven data.

1. Log in to Tier0, and then select **UNS > Event Flow**.
2. Click **New Event Flow**, enter a name and click **Save**.
3. Click the flow you just added, drag an `mqtt in` node, and enter the configurations.
 - Tier0 MQTT broker address: `emqx:1883`
 - Topic: The topic to obtain data from, for example, `factory/workshop/order`.



The screenshot shows the 'Edit mqtt in node' configuration window. It includes a title bar with 'Edit mqtt in node' and buttons for 'Delete', 'Cancel', and 'Done'. The main area is titled 'Properties' and contains several configuration fields: 'Server' (a dropdown menu showing 'emqx'), 'Action' (a dropdown menu showing 'Subscribe to single topic'), 'Topic' (a text input field containing 'factory/workshop/order'), 'QoS' (a dropdown menu showing '2'), 'Output' (a dropdown menu showing 'auto-detect (parsed JSON object, string or buf)'), and 'Name' (a text input field containing 'Name').

4. Drag a `function` node to the canvas, and write a script to format the data obtained according to third-party receiver API.

```
// parse payload if it's a string
if (typeof msg.payload === "string") {
  msg.payload = JSON.parse(msg.payload);
}

// set headers
msg.headers = {
  "Content-Type": "application/json"
};

return msg;
```

5. Use an `http request` node to send the data to the third party through API.

Edit http request node

Delete Cancel Done

Properties

Method POST

URL http://thirdparty.api.url/data/destination

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a UTF-8 string

Headers

+ add

Name Name

6. Deploy the flow, and data will be pushed to the third party when the topic data changes.

Obtaining Data Directly from Tier0 Database

info

This method is suitable for requests on history data.

1. Log in to Tier0, and then select **UNS > Event Flow**.
2. Click **New Event Flow**, enter a name and click **Save**.
3. Click the flow you just added, drag an `inject` node.
4. Drag a `postgres` node to the canvas, and connect to Tier0 PostgreSQL database.
 - o **Connection**
 - Host: `postgres1`
 - Port: `5432`

- Database: postgres
- SSL: false
- **Security**
 - User: postgres
 - Password: postgres

Edit postgresql node > Edit postgresSQLConfig node

Delete Cancel Update

Properties

Name dbConnection

Connection Security Pool

Host postgresql

Port 5432

Database postgres

SSL false

Edit postgresql node > Edit postgresSQLConfig node

Delete Cancel Update

Properties

Name dbConnection

Connection Security Pool

User postgres

Password

5. Write SQL statements to query data from Tier0 database.

info

Get the table name from topic **Details** under **Namespace**.

Namespace

factory / workshop / order

Topic Template Label

Please enter name/alias

Equipment(1)

CNC

factory(1)

workshop(1)

order

fggg

order

Details

Topic factory/workshop/order

Alias _order_0baa755c15544f81a302

Display Name

Description

Reference Template

Data Type Relational

```
SELECT * FROM _order_0baa755c15544f81a302;
```

Edit postgresql node

Delete Cancel Done

Properties

Name

Server

☐ Split results in multiple messages

Number of rows per message

Query

```
1 SELECT * FROM _order_0baa755c15544f81a302;
```

6. Add an `http request` or `mqtt out` node to transmit the queried data to the third party.

Advanced Guide

Container Management

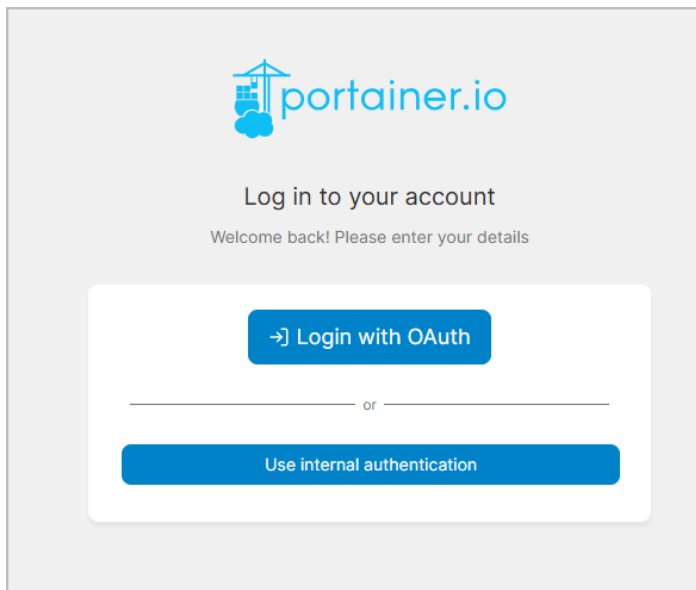
Tier0 uses Portainer to manage containers, making it easy to deploy, manage, and scale applications.

info

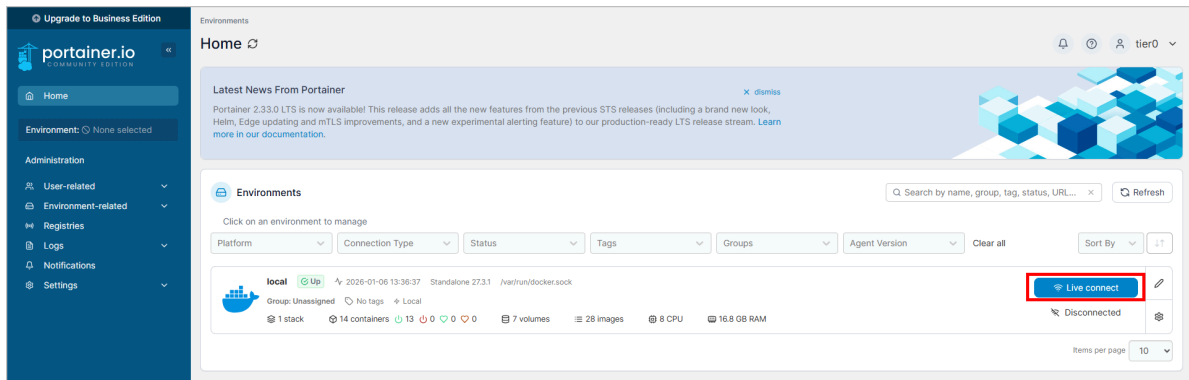
In this guide, simple operations that relate to general container management are introduced. For detailed Portainer usage, see [Portainer Documentation](#).

Accessing Container Management

1. Log in to Tier0, and then select **Dev Tools > Containers**.

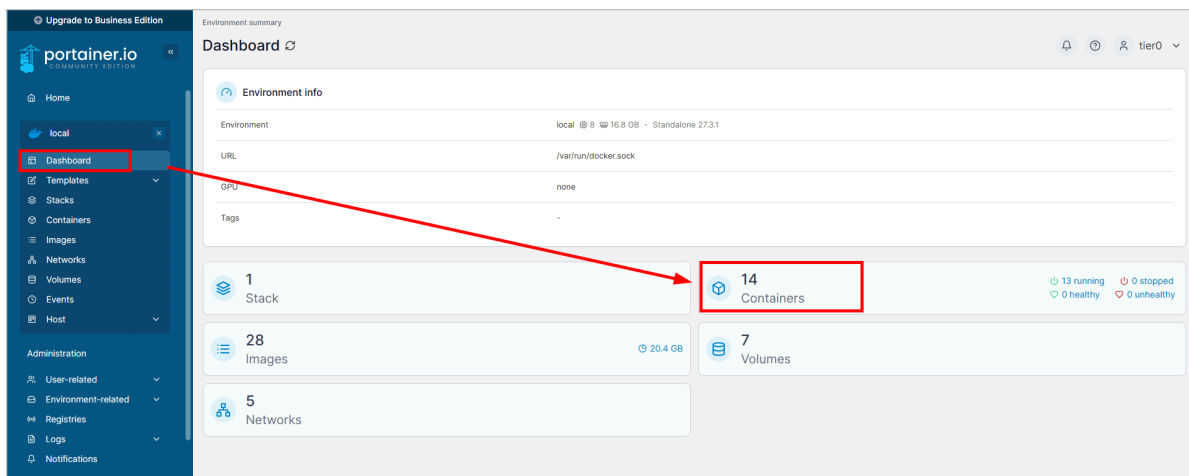


2. Click **Login with OAuth**, and then click **Live connect** to start managing containers.

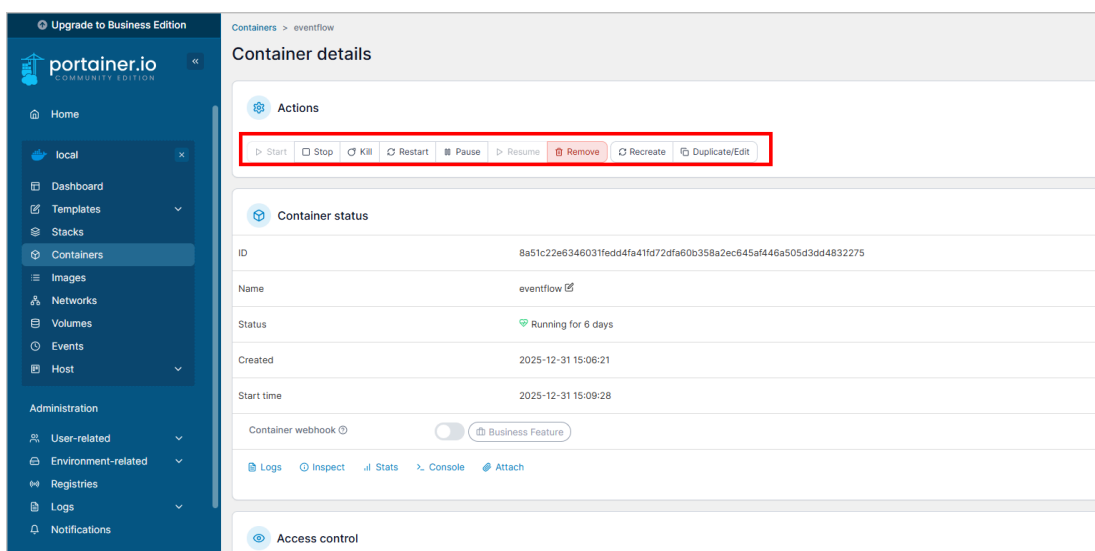


General Operations on Containers

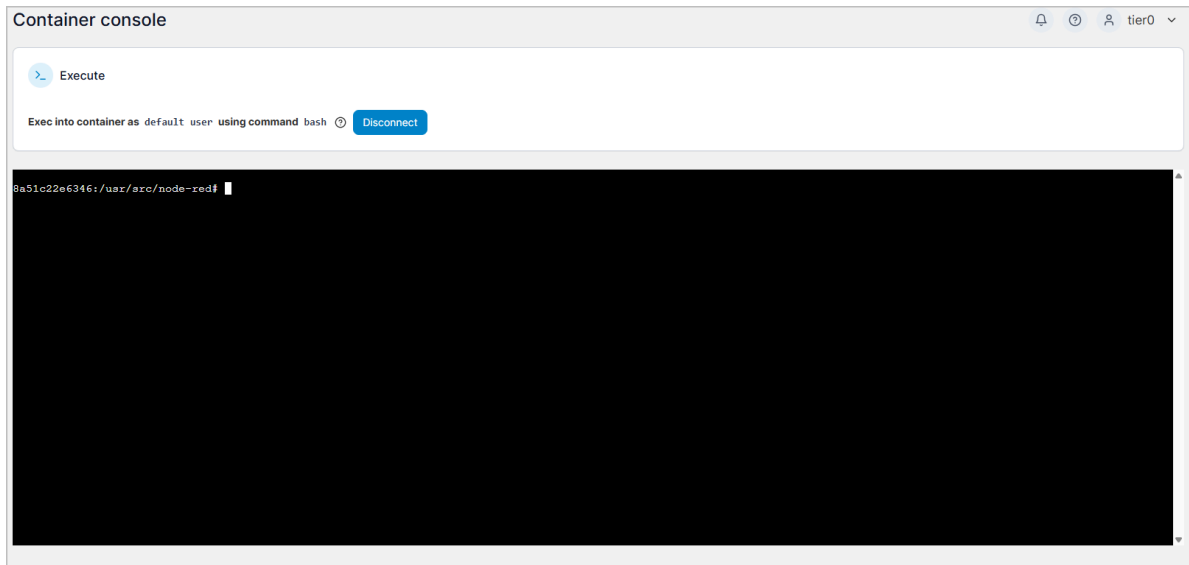
1. On the dashboard, click **Containers**.



2. Select a container, and then you can start, stop, restart, pause, or remove the container.

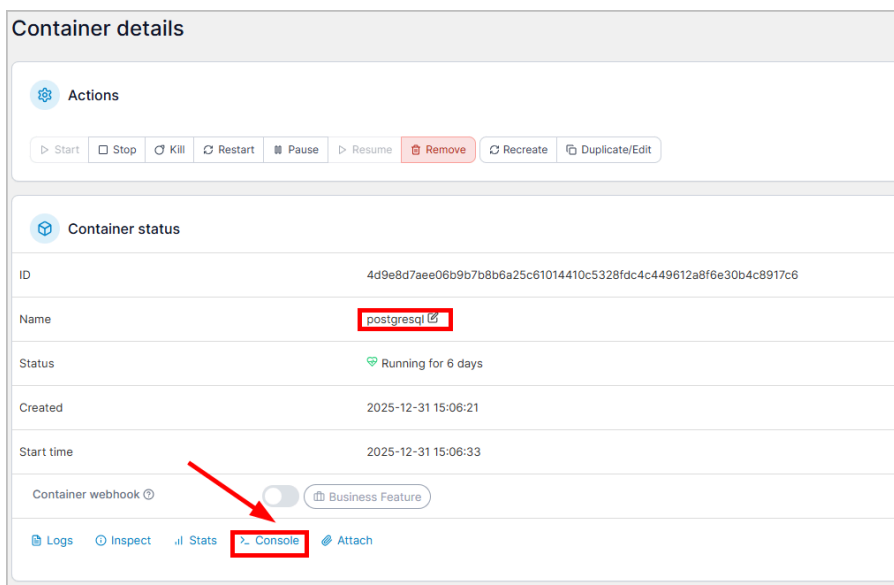


3. (Optional) Click **Console** to access the container terminal for more complex operations.



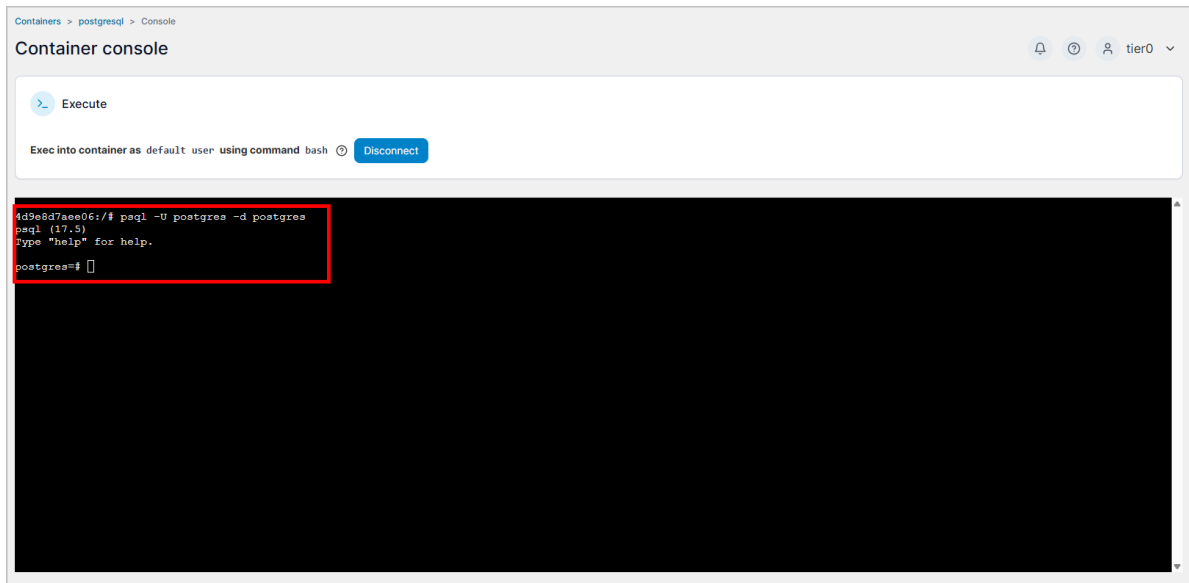
Accessing Database on Container

1. On the dashboard, click **Containers**.
2. On page 2, select **postgresql**, and then click **Console**.



3. Click **Connect**, and then use the following command to access the database.

```
psql -U postgres -d postgres
```

Notebook

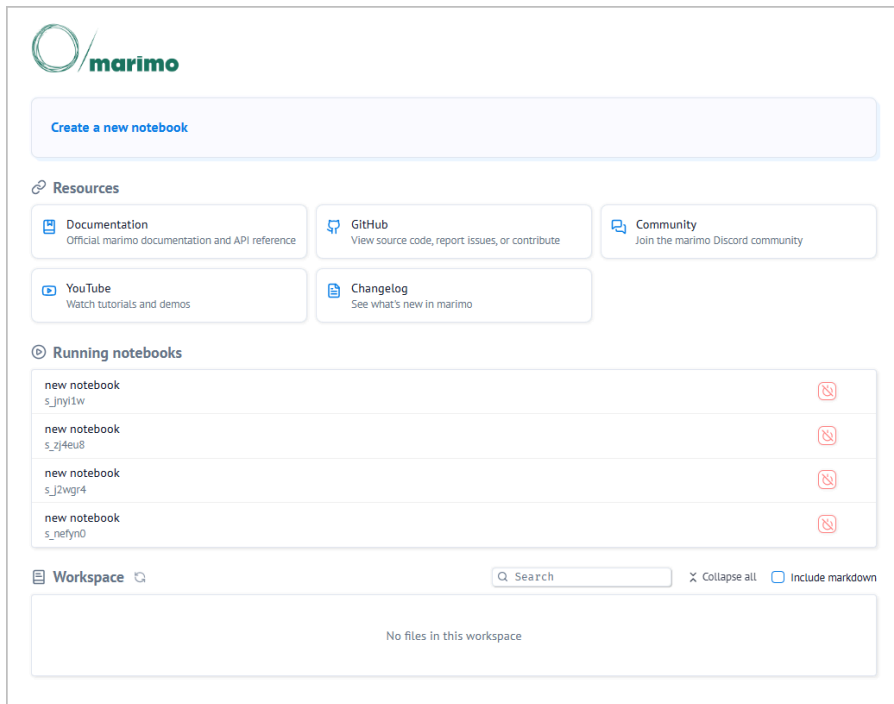
Tier0 adopts Marimo for easy python script usage. For example, you can train models, run SQL queries and design dashboards with data connected in **Namespace**.

Using Notebook to Run SQL Queries

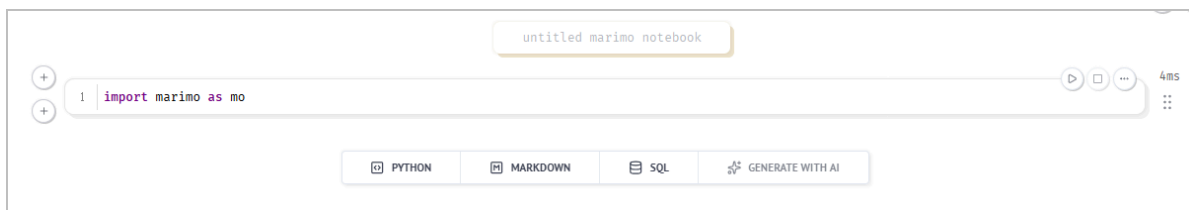
note

Tier0 databases are initially connected.

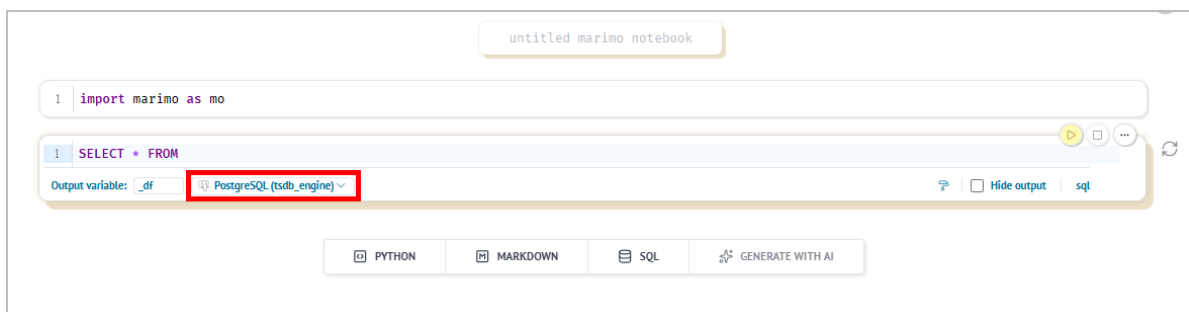
1. Log in to Tier0, and then select **Dev Tools > Notebook**.



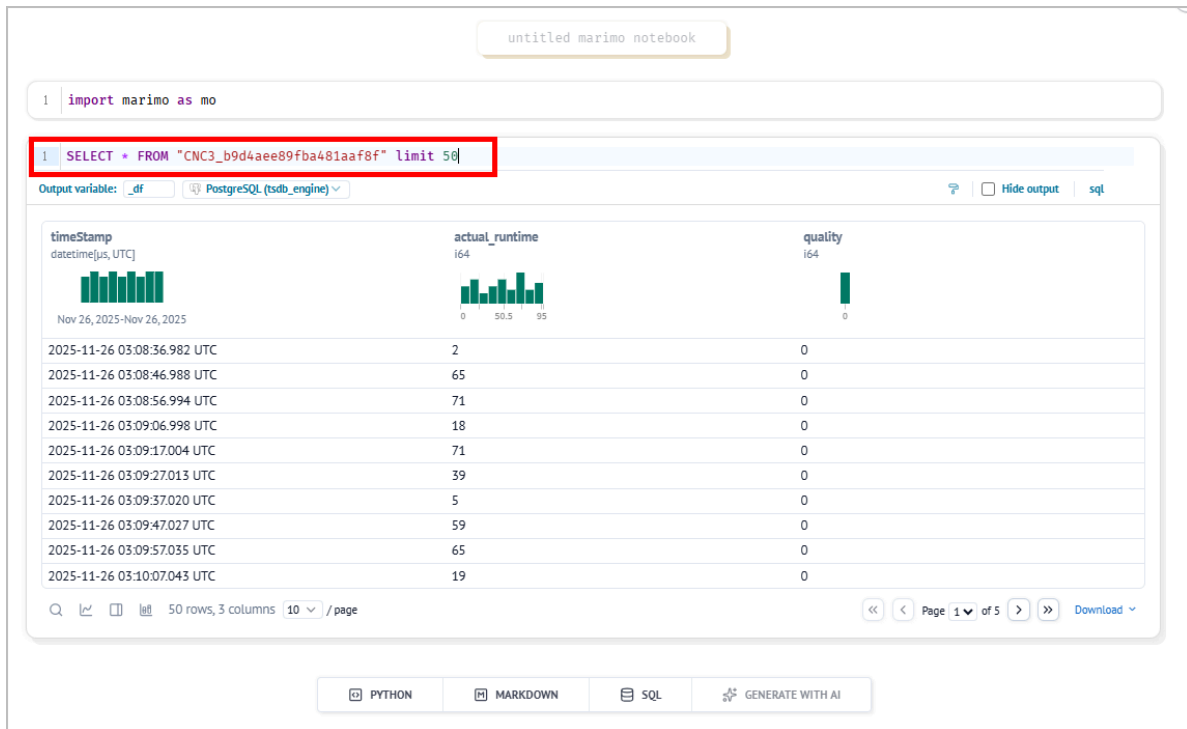
2. Click **Create a new notebook**, and then click  to run the first cell.



3. Click **SQL** to add a new cell, and then select a connected database to run SQL statements.



4. Get the table name from topic **Details** under **Namespace**, and then write SQL statements.



✓ tip

- Click  on the left panel to view the list of connected databases.
- Find the table you need, click  next to the table name to generate SQL statements automatically.

Using Notebook to Draw Charts

1. Log in to Tier0, and then select **Dev Tools > Notebook**.
2. Click **Create a new notebook**, and then add the following 3 cells to draw a simple line chart.

- Cell 1: Import libraries

```
import marimo as mo
```

- Cell 2: Query data from the database

```
_df = mo.sql(  
    f"""  
    SELECT *  
    FROM "table_name"  
    LIMIT 100  
    """,  
    engine=tsdb_engine  
)  
data = _df
```

✓ **tip**

Use a SQL cell to write queries and then click  on the cell end to switch the cell to Python. Make sure you assign the query result to a variable (e.g. `data`).

- Cell 3: Draw a line chart

```
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10,5))  
plt.plot(data["actual_runtime"])  
plt.title("Actual Runtime")  
plt.xlabel("Index")  
plt.ylabel("actual_runtime")  
plt.grid(True)  
plt.show()
```

3. Click  at the lower-right corner to run all cells and view the chart.

untitled marimo notebook

1 import marimo as mo

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

1

2

3

4

5

6

7

8

9

timeStamp

datetime[us,UTC]

actual_runtime

i64

quality

i64

Nov 26, 2025-Nov 26, 2025

0

46

93

0

2025-11-26 03:08:36.982 UTC	2	0
2025-11-26 03:08:46.988 UTC	65	0
2025-11-26 03:08:56.994 UTC	71	0
2025-11-26 03:09:06.998 UTC	18	0
2025-11-26 03:09:17.004 UTC	71	0
2025-11-26 03:09:27.013 UTC	39	0
2025-11-26 03:09:37.020 UTC	5	0
2025-11-26 03:09:47.027 UTC	59	0
2025-11-26 03:09:57.035 UTC	65	0
2025-11-26 03:10:07.043 UTC	19	0

Q

🔍

📄

📊

100 rows, 3 columns

10

/ page

⏪

⏩

Page 1

of 10

⏪

⏩

Download

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

1

2

3

4

5

6

7

8

9

10

11

Actual Runtime

100

80

60

40

20

0

0

20

40

60

80

100

actual_runtime

Index

PYTHON

MARKDOWN

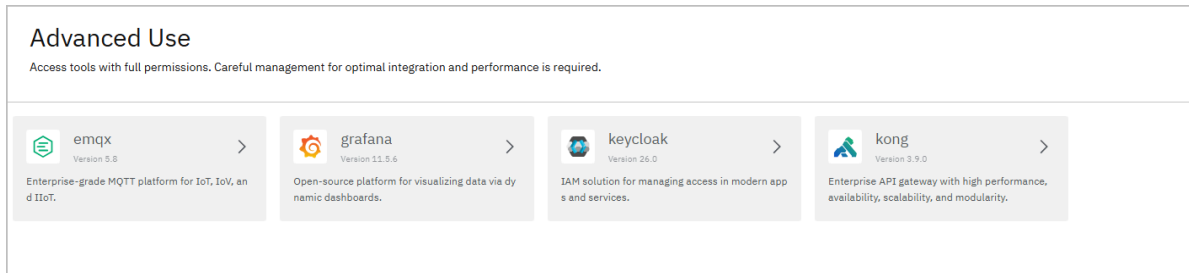
SQL

GENERATE WITH AI

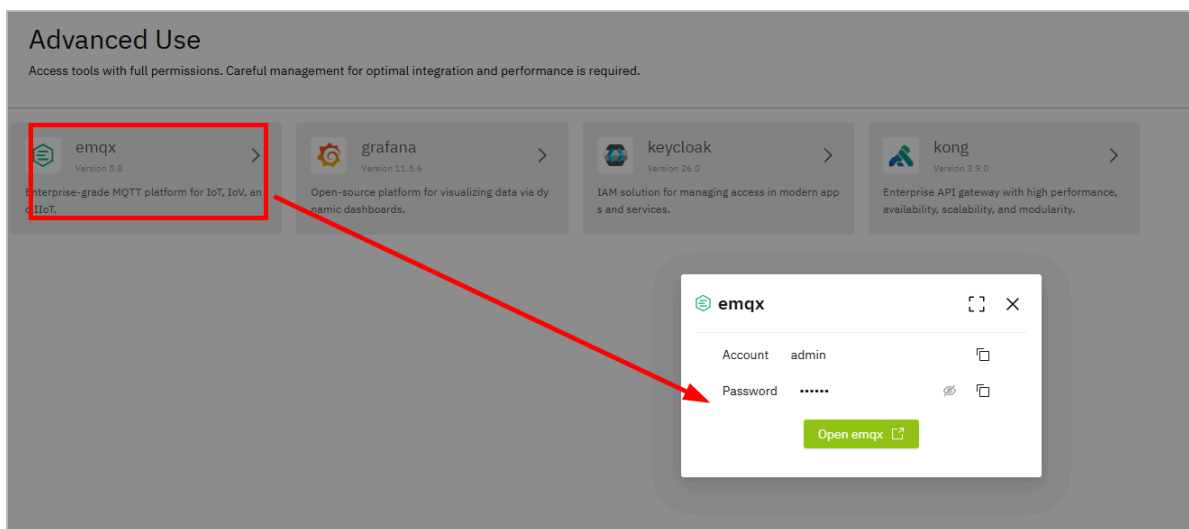
Advanced Use

Tier0 empowers you to use open source products that are closely integrated with it, with full permissions.

1. Log in to Tier0, and then select **Advanced Use** under **Dev Tools**.



2. Click an open source product to access its management page.



System

Routing Management

Tier0 supports service routing based on Konga, and you can add your application to Tier0 for integrated entry. Following certain rules, you can:

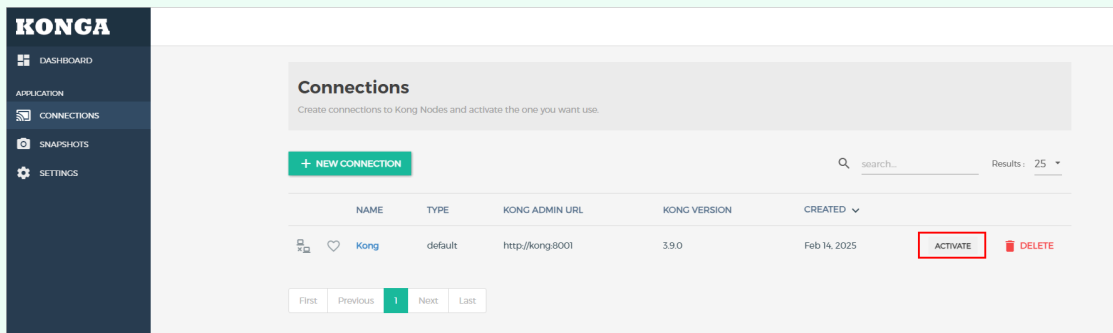
- Set the routing path to your backend service.
- Fashion access authorization mechanisms.
- Add external service to Tier0 frontend as one of the applications.

Creating Service

1. Select **System > Routing Management**, and then click **SERVICES**.

✓ tip

If the **DASHBOARD** shows no active connection, go to **CONNECTIONS**, and then click **ACTIVATE** next to **Kong**.



2. Click **ADD NEW SERVICE**, and then enter the information of your backend service.

i info

These parameters are necessary. For instance, you have a backend resource <https://example.com:8080>.

- **Protocol:** https
- **Host:** example.com
- **Port:** 8080
- **Path:** Request path prefix, which will added to the path of request from client before the request gets forwarded to your service.

CREATE SERVICE×

Name <i>(optional)</i>	 The service name.
Description <i>(optional)</i>	 An optional service description.
Tags <i>(optional)</i>	 Optionally add tags to the service
Url <i>(shorthand-attribute)</i>	 Shorthand attribute to set protocol , host , port and path at once. This attribute is write-only (the Admin API never "returns" the url).
Protocol <i>(semi-optional)</i>	 The protocol used to communicate with the upstream. It can be one of http or https .
Host <i>(semi-optional)</i>	 The host of the upstream server.
Port <i>(semi-optional)</i>	 The upstream server port. Defaults to 80 .
Path <i>(optional)</i>	 The path to be used in requests to the upstream server. Empty by default.
Retries <i>(optional)</i>	5 The number of retries to execute upon failure to proxy. The default is 5 .

3. Click **SUBMIT SERVICE**.

Building Routes

Build the routing path for accessing the service.

1. On the **Service Details** page, click **Routes**.
2. Click **ADD ROUTE**, and then enter the information of the route to access your service.

- **Tags:** We have mapped some tags with fixed meanings.

Tag	Description
menu	The service will appear as an application on Tier0 homepage.
parentName:menu.tag.xxx	xxx is the name of an existing first-level menu in Tier0 (all in lowercase). It indicates that the current route appears as a second-level menu of xxx.
homeParentName:menu.tag.xxx	xxx is the name of an existing first-level menu in Tier0 (all in lowercase). It indicates that the current route appears in the navigation bar and serves as a second-level menu of xxx.
description:menu.desc.xxx	xxx is the name of the route. It represents the description information corresponding to xxx.

- **Paths:** Request paths that will be recognized by Kong and Kong will match and forward the request to your service.

ADD ROUTE TO MY-BACKEND
X

* For hosts, paths, methods and protocols, snis, sources, headers and destinations press enter to apply every value you type

Name (optional)	 The name of the Route.
Tags (optional)	 Optionally add tags to the route
Hosts (semi-optional)	 A list of domain names that match this Route. For example: example.com. At least one of hosts, paths, or methods must be set.
Paths (semi-optional)	 A list of paths that match this Route. For example: /my-path. At least one of <code>hosts</code> , <code>paths</code> , or <code>methods</code> must be set.
Headers (semi-optional)	 One or more lists of values indexed by header name that will cause this Route to match if present in the request. The <code>Host</code> header cannot be used with this attribute: hosts should be specified using the <code>hosts</code> attribute. Field values format example: <code>x-some-header:foo,bar</code>
Path handling	v1 Controls how the Service path, Route path and requested path are combined when sending a request to the upstream. See above for a detailed description of each behavior. Accepted values are: <code>"v0"</code> , <code>"v1"</code> . Defaults to <code>"v1"</code> .
Https redirect status code (optional)	 426 The status code Kong responds with when all properties of a Route match except the protocol, i.e. if the protocol of the request is <code>HTTP</code> instead of <code>HTTPS</code> . <code>Location</code> header is injected by Kong if the field is set to 301, 302, 307 or 308. Defaults to <code>426</code> .

3. Click **SUBMIT ROUTE**.

Setting Authentication

Set authentication mechanisms for your service.

1. On the **Service Details** page, click **Plugins**.
2. Click **ADD PLUGIN**, and select **Key Auth**.

📘 note

You can add other authentication methods. For more details, see [Konga](#).

3. Enter the **Key names** and decide how the authentication information should be transmitted.

Add Key Authentication (also referred to as an API key) to your APIs. Consumers then add their key either in a querystring parameter or a header to authenticate their requests.

consumer

The CONSUMER ID that this plugin configuration will target. This value can only be used if authentication has been enabled so that the system can identify the user making the request. If left blank, the plugin will be applied to all consumers.

key names

Tip: Press **Enter** to accept a value.

Describes an array of comma separated parameter names where the plugin will look for a key. The client must send the authentication key in one of those key names, and the plugin will try to read the credential from a header or the querystring parameter with the same name.

hide credentials

NO

An optional boolean value telling the plugin to hide the credential to the upstream API server. It will be removed by Kong before proxying the request.

anonymous

key in header

YES

key in query

YES

key in body

NO

run on preflight

YES

realm

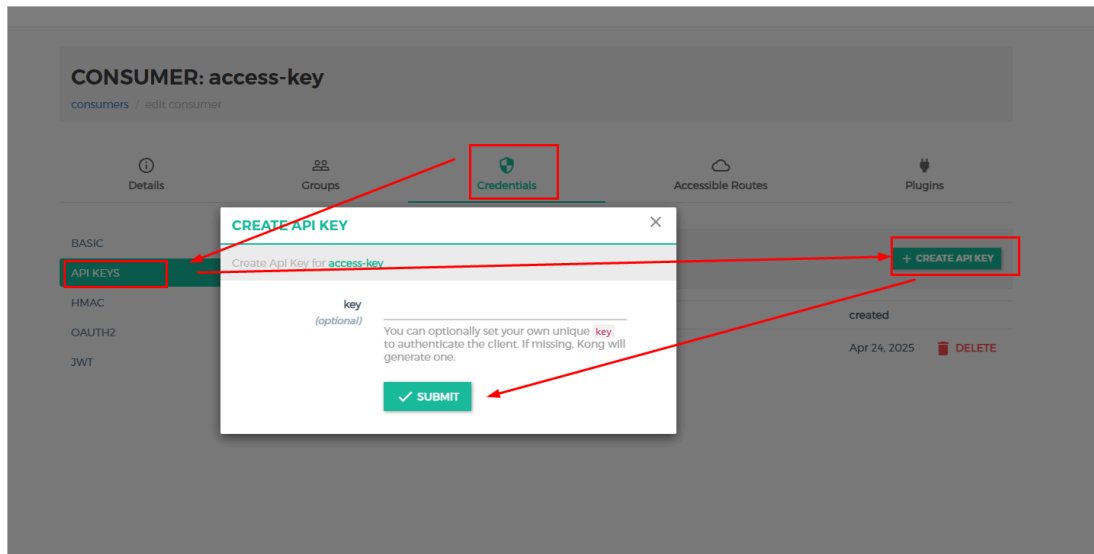
4. Click **ADD PLUGIN**.

5. Click **CONSUMERS** on the left side menu to add an authentication key.

1. Click **CREATE CONSUMER**, and give it a username or ID.

2. On the details page, select **Credentials**.

3. Select **API KEYS**, and then create an API key.



4. Leave it empty and click **SUBMIT**, Konga will generate a custom key, which is necessary when accessing your service.

User Management

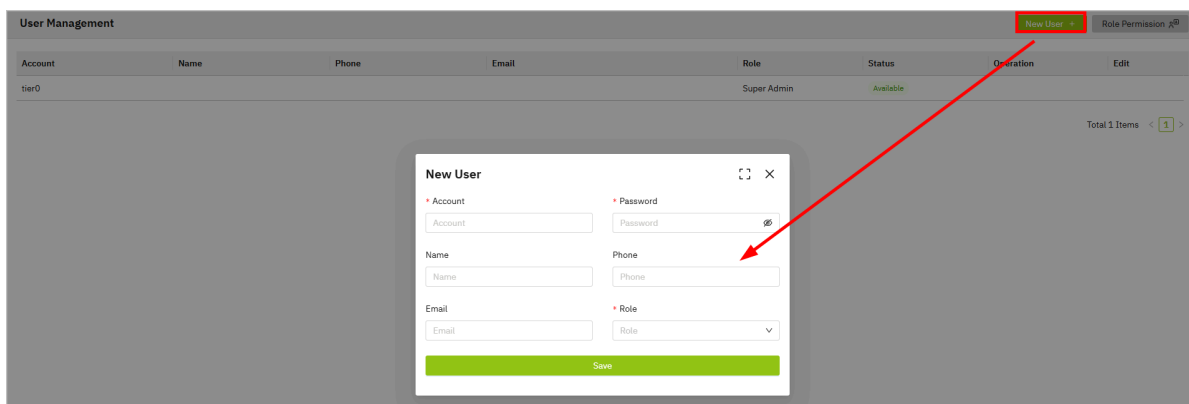
In Tier0, you can add users and assign different roles to them, easily managing module access. Meanwhile, you can also allow users to register account by themselves.

info

We adopted **Keycloak** to achieve comprehensive user management. You can customize it under **System > Authentication**.

Adding User

1. Log in to Tier0, and then select **User Management** under **System**.
2. Click **New User** at the upper-right corner.



3. Enter the information and select a role for the user, and then click **Save**.

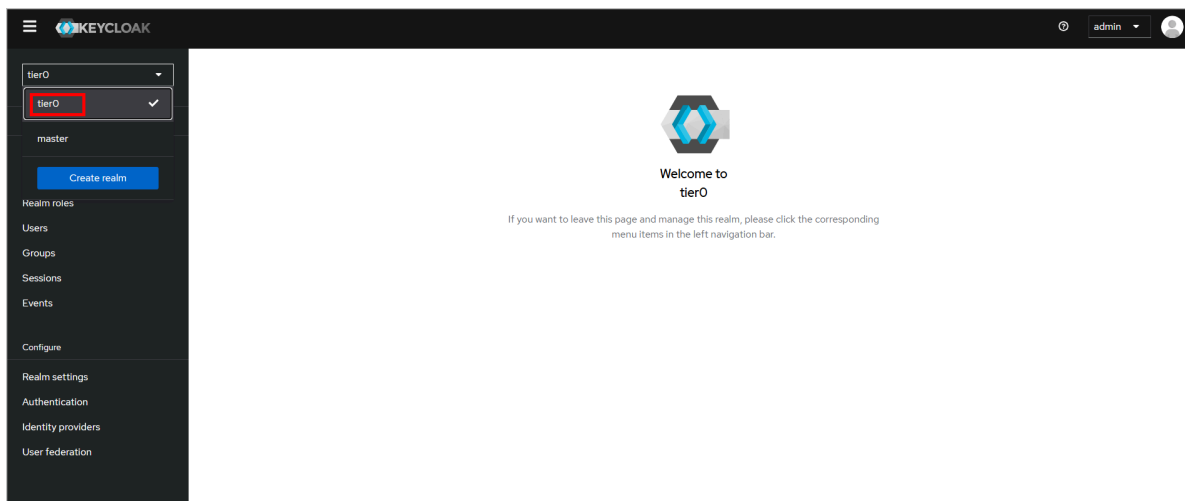
Account	Name	Phone	Email	Role	Status	Operation	Edit
tier0				超级管理员	Available		
test				一般用户	Available	Disable	

- You can disable, edit, delete user and reset password of it.
- The default user cannot be disabled or deleted.

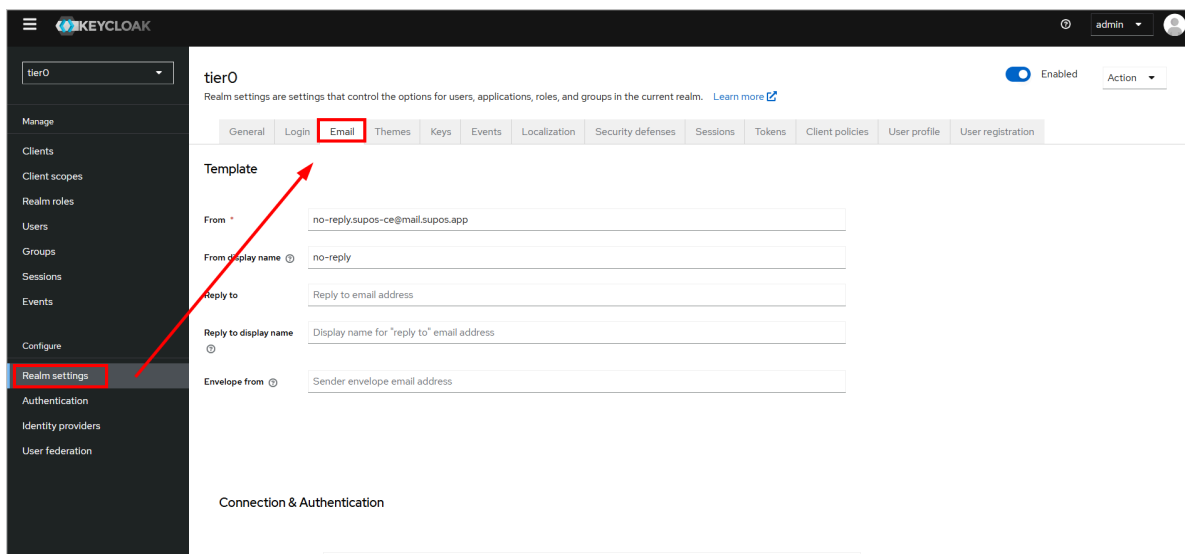
Registering User

For user registration, email is necessary. You need to set up an Email server for sending validation email.

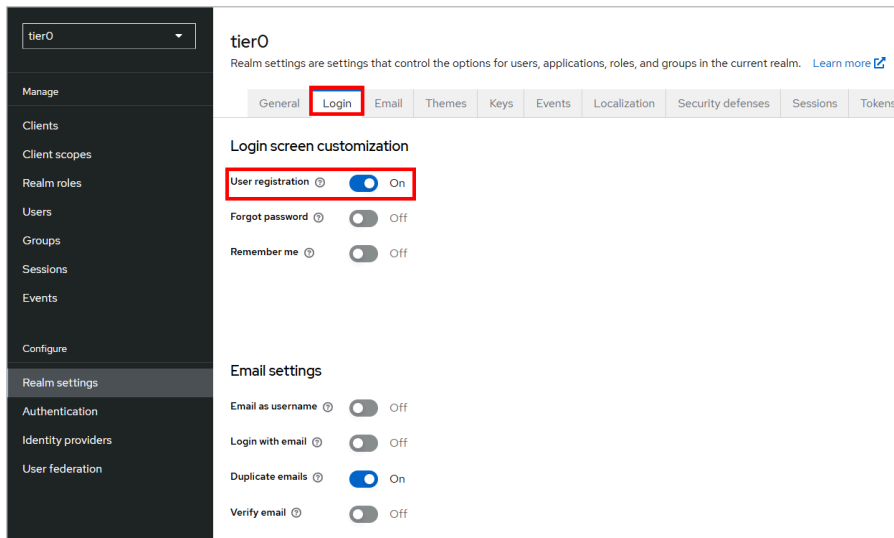
1. Select **System > Authentication**, and then log in to Keycloak with **admin/tier0**.
2. Switch the realm to **tier0**, and then select **Realm settings**.



3. Click **Email**, and then enter the email template and server information.



4. Save the settings, and then click **Login** to enable **User registration**.



5. Go to the Tier0 login page, and then click **Register**.

6. Enter the information to apply for a user account.



Open Data

How to Access Data in Tier0

Tier0 provides 3 types of data subscription and invocation methods: API, MQTT, and MCP.

API

To access data through API, you need to use the URL and secret key provided in the platform.

1. Log in to Tier0, and select **System > Open Data**.
2. Click **New Key** to create a secret key.

App Key	Creation Time	Status	Actions
ab91bf4e-8a0e-42fb-bcd...	16:37:16 08/04/2026	Enable	Disable

Built-in

RestAPI MQTT MCP Server

REST API Access Method

REST API URL: https://enterprise-dev.tier0.dev/open-api

Secret Key: ab91bf4e-8a0e-42fb-bcd...

Call Instructions: 1. Apply for Secret Key, 2. Secret key must be added in request headers.

3. Use the URL displayed on the bottom, and the created key to obtain data from Tier0.

info

Click **API List** to read the API documentation.

MQTT

You can use third-party clients like MQTTX, MQTT Explorer, or custom code to subscribe to data through MQTT.

1. Log in to Tier0, and select **System > Open Data**.
2. Click **MQTT** to view the MQTT connection information, including the broker address, port and topic.

MQTT Access Method

JS JAVA

MQTT URL: mqtt://enterprise-dev.tier0.dev

MQTT Port: 1883

Topic: demo

Dependency: npm install mqtt

Payload: { 'timestamp': 1775723122147, 'aa': '123.456789', 'quality': 23 }

```
const mqtt = require('mqtt');
const options = {
  clean: true,
  connectTimeout: 4000,
  clientId: 'emqx_test',
  rejectUnauthorized: false,
};
const connectUrl = 'ws://enterprise-dev.tier0.dev:8083/mqtt';
const client = mqtt.connect(connectUrl, options);
client.on('connect', function () {
  console.log('Connected');
  client.subscribe('demo', function (err) {
    console.log(err);
  });
  if (!err) {
    client.publish('demo', JSON.stringify({ 'timestamp': 1775723122147, 'aa': '123.456789', 'quality': 23 }));
  }
});
client.on('message', function (topic, message) {
  console.log(topic, message.toString());
});
```

Show Less

info

Tier0 provides script samples in both JavaScript and Java to help you get started.

MCP Server

Tier0 enables you to use an MCP server to get data from the platform through a custom LLM.

1. Log in to Tier0, and select **System > Open Data**.
2. Click **MCP Server** to view the MCP connection details and example.

RestAPIMQTTMCP Server

MCP Server Access Method

Install

npm install -g @sup-platform/mcp-server

Command Line Param

MCP Server supports the following command line parameters:

Parameter	Description	
--port	Server listening port.	3000
--transport	Transport type. Stdio and streamer are supported.	stdio
--supos-api-url	Base URL for the Tier0 API.	-
--supos-api-key	Access key for the Tier0 API.	-
--openapi-path	Path to the OpenAPI specification file.	{supos-ap

Environment Variable

Environment variables, MCP Server can be configured through en...

Variable	Description	Corresponding Command Line Param
	Base URL for the Tier0 API.	--supos-api-url
	Access key for the Tier0 API.	--supos-api-key
	Path to the OpenAPI specification file.	--openapi-path

Desktop App

Integration

To integrate MCP Server into a desktop application, add the following to the application server configuration (Cline plugin as an example):

```
{  "mcpServers": {    "mcp-server-supos-stdio": { // Stdio Transport Mechanism      "disabled": false,      "timeout": 60,      "type": "stdio",      "command": "npx",      "args": ["-y", "@sup-platform/mcp-server"],      "env": {        "SUPOS_API_URL": "xxx",        "SUPOS_API_KEY": "xxx",        "SUPOS_OPEN_PATH": "xxx"      }    },    "mcp-server-supos-streamable": { // Streamable HTTP Transfer Mechanism      "timeout": 60,      "url": "http://localhost:3000/mcp", // Streamable Server Listening Address      "type": "streamableHttp"    }  }}
```

info

- Cline plugin usage is used as an example.
- The required URL and key can be obtained on the **RestAPI** tab.
- You can click **API List** on the **RestAPI** tab, and copy the address as the **SUPOS_OPEN_PATH** in the script to help AI understand.

Menu Config

Tier0 enables basic configuration on function modules in the style of menus, including module position, visibility, and detailed information. Meanwhile, it can be used as an application manager where you can add and configure your web app on Tier0.

Adding Menu

1. Log in to Tier0, and then select **System > Menu Config**.

Menu Config

Menu List

Home

Overview

UNS

Namespace

Source Flow

Event Flow

Dashboards

Dev Tools

SQL Editor

Notebook

Data Source Management

App Space

Doc

System

Routing Management

Authentication

Namespace level 2

Save

Basic Info

Menu Source

Route Acquisition

Namespace

Menu Code

Namespace

Menu Name

Namespace

Menu Icon

Picture size: 28*28

Open with

☒ Open Current Page
 ☐ Open New Page

Menu URL

/uns

Menu Description

Build data models with clear structure.

Homepage Display

☒

Operation Info

Operation Code

Operation Name

Operation Desc

Operation

+ New Operation Code

No.	Name	Description
1	Operation	Add menu in floder/file structure and refresh the menu list.
2	List	Lists existing structured menus.
3	Information	Displays information of the selected menu.

2. Click  on the top to add a menu folder.

3. Enter the folder information and then click **Save**.

test New level 1

Basic Info

* Menu Code

test

* Menu Name

test

Menu Icon

Picture size: 28*28

Menu Description

Homepage Display

☒

info

Menu code cannot be duplicated.

4. Select the created folder, and then click  to add a menu file.
5. Enter the menu information and then click **Save**.

New level 2 Save

Basic Info

* Menu Source

Manual

* Menu Code

* Menu Name

Menu Icon

+

Picture size: 28*28

Open with

☐ Open Current Page

☒ Open New Page

* Menu URL

Menu Description

Homepage Display

☒

Parameter	Description
Menu Source	Select menu source between manual addition and existing options.
Menu Code	Unique identification of the menu.
Open with	Directly open the menu on the current page or on a new page.
Menu URL	Access address of the added function module (menu).
Homepage Display	Whether or not to display the menu on the homepage.

6. (Optional) Go back to **Home** to check the added menu if it is set to display on homepage.

Editing Existing Menus

1. Log in to Tier0, and then select **System > Menu Config**.
2. Select a menu from the list, edit its information on the right side, and then click **Save**.

Namespace
level 2
Save

Basic Info

Menu Source
Route Acquisition
Namespace

Menu Code
Namespace

Menu Name
Namespace

Menu Icon

Picture size: 28*28

Open with
☒ Open Current Page
☐ Open New Page

Menu URL
/uns

Menu Description
Build data models with clear structure.

Homepage Display
☒

Operation Info
+ New Operation Code

Operation Code	Operation Name	Operation Desc	Operation
Namespace.uns_import	UNS-Import		
Namespace.uns_export	UNS-Export		
Namespace.uns_batch_generation	UNS-Batch Generation		
Namespace.label_detail	Label-Edit		
Namespace.label_delete	Label-Delete		
Namespace.label_add	Label-New		
Namespace.folder_detail	Path-Edit		
Namespace.folder_delete	Path-Delete		
Namespace.folder_copy	Path-Copy		
Namespace.folder_add	Path-New		
Namespace.file_detail	Topic-Edit		
Namespace.file_delete	Topic-Delete		
Namespace.file_copy	Topic-Copy		

⚠ warning

For original menus provided by Tier0, do not change the menu source, URL and operation code. Otherwise the module may not work properly.

Managing Menu Position and Visibility

- On the left list, drag a menu to change its position in the folder/file structure, and it will show on the homepage automatically.
- Click  next to a menu to hide it from homepage.
- Click  to delete a menu.

⚠ warning

- Deleting a folder will delete all menus under it.
- Deleting a menu cannot be restored. Please proceed with caution.

Glossary

Key Terms and Definitions

Term	Definition
Tier0	An open-source industrial data integration platform built on the Unified Namespace (UNS) methodology, designed for managing, processing, and optimizing industrial data.
UNS (Unified Namespace)	A data management method that centralizes data from various devices and systems, providing a structured and unified framework to prevent data silos.
Node-RED	An open-source automation tool that enables users to wire together devices, APIs, and online services using a flow-based programming approach.
Grafana	An open-source platform for monitoring and data visualization, which allows users to create dashboards to view real-time data from various sources.
MQTT	A lightweight messaging protocol for small sensors and mobile devices optimized for high-latency or unreliable networks, commonly used for IoT communication.
JSON	JavaScript Object Notation, a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.
TimescaleDB	An open-source time-series database built on PostgreSQL, designed for handling large-scale time-series data, especially useful for monitoring systems and IoT applications.
PostgreSQL	An advanced open-source relational database management system that uses SQL for querying and is designed for high performance and reliability.
API (Application Programming Interface)	A set of protocols and tools that allows different software applications to communicate and interact with each other.
Topic	A unique identifier for data in the Tier0 system that helps classify and organize data from various sources for easy access and management.

Term	Definition
OEE (Overall Equipment Effectiveness)	A metric used to assess the efficiency and effectiveness of equipment in a manufacturing environment, considering availability, performance, and quality.
AGV (Automated Guided Vehicle)	An automated vehicle used in manufacturing or warehouse environments to transport materials, reducing human intervention in logistics operations.
REST API	A web service interface that allows applications to communicate over HTTP, using standard methods like GET, POST, PUT, and DELETE to access resources.
Time-Series Data	Data that is collected and recorded at regular intervals over time, such as equipment run time, temperature, and other performance metrics.
JSON Payload	Data transferred in the form of a JSON object that contains relevant information, such as sensor readings or device states, to be processed or stored.
Authentication	The process of verifying the identity of a user or system to ensure that only authorized entities can access resources.
Event-Driven	A programming paradigm in which the flow of the program is determined by events, such as user actions, sensor data, or messages from other systems.
JSONB	A PostgreSQL data type that stores JSON data in a binary format, offering efficient indexing, faster querying, and improved performance compared to plain JSON fields.